AP Computer Science A Notebook
2025-2026

Pablo Silva

Unit 0 - Getting Started

0.1 - Preface

- No notes to show, all contributions.

0.2 - About the Exam

- It is 3 hours long, has two sections.
    - Multiple-choice: 1 hr 30 mins, 55% of grade, 42 questions.
    - Free response: 1 hr 30 mins, 45% of grade, 4 questions.
- Access to the AP CSA Java Quick Reference Sheet throughout the exam.
- Shorter FRQs than previous years, total 25 points.
    - FRQ 1 - Methods and control structures. (7 pts)
        - Write two methods or one constructor and one method of a class.
        - Part A (4 pts) - Method/constructor will be iterative or conditional statements, or both.
        - Part B (3 pts) - Method/constructor will require calling String methods. (different from FRQs prior to 2026)
    - FRQ 2 - Class Design (7 pts)
        - Design and implement a class based on specifications + examples.
        - Scenario and associated examples.

        - Must have a class header, instance variables, a constructor, a method, and implementation of the latter 2. (inheritance no longer tested)
    - FRQ 3 - Data Analysis with ArrayList (5 pts)
        - Scenario and associated classes.
        - Write one method based on a class and provide specifications + examples.
        - Required to use, analyze, and manipulate data in an ArrayList structure (actual arrays will not be tested in an FRQ.)
    - FRQ 4 -2D Array (6 pts)
        - Scenario and associated classes.
        - Write one method based on a class and provide specifications + examples.
        - Required to use, analyze, and manipulate data in a 2D array structure. (usually requires nested loops)
- Personal Progress Checks are given for each unit throughout the course, and you also must have at least 20 hours of lab time to practice Java programming.

0.3 - Transitioning from AP CSP to AP CSA

- If you've taken AP CSP, you will be well prepared for AP CSA.
- Differences from Python or block programming (common in CSP):
    - Spell things correctly and consistently.
    - Pay attention to case (lower-case, upper-case), Java is case-sensitive.
    - Punctuation is important. Most statements in Java end with a semicolon (;).
    - Java uses curly braces ({}) to group statements together.
    - Indentation is still required.

0.4 - Java Development Environments

- In order to compile a Java source file into a Java class file, we used **compilers**.
- Most compilers use an **Integrated Development Environment** (IDE) that has a compiler built in → write, compile, run and debug programs.
- Learn java by using the interactive coding panels called **Active Code** in the e-book.
    - It remembers your changes and a history of them as well.
- Use Java IDEs for things like user input.
    - I highly recommend setting one up at VSCode.
    - I used a flash drive in order to transfer my data from my home computer to Chromebook (through web VSCode) or Schultz's laptops.
    - Read 0.4.3 to install VSCode.

0.5 - Growth Mindset and Pair Programming

0.5.1 - Growth Mindset

- It is important to have a positive attitude when coding.
- You will face problems and failures, it's part of the process.
- Note your mistakes and make a plan to overcome them.
- Use common sense.

0.5.2 - Pair Programming

- **Pair programming** is a successful software development technique in which two programmers work together at one computer.
- One type of code (driver) while the other (navigator) gives ideas and feedback.
- Buddy programming is similar, in which you have two computers and you help each other out.
- Switch in-between roles → produce better code overall.

0.6 - Short PD Pretest

- Personal note: at a first-glance, Java seems a lot like C++. The only questions that I think I missed were anything that were related to classes and public/private methods of said classes.

0.7 - Pretest for the AP CSA Exam

- As I mentioned above, roughly 80% of these questions could've been answered from just knowing C++ alone. I will say though, classes seem to be a huge focus of Java.

0.8 - Survey

- No notes, just a survey.

Unit 1 - Using Objects and Methods

1.1 - Introduction to Algorithms, Programming, and Compilers

1.1.1 - Algorithms

- **Algorithms** are step-by-step processes that complete a task or solve a problem.
    - Used in many areas of life.
    - In computer science, they are used to solve problems and create software.
        - Sort a list of numbers, search a word in a document, or code a game.
    - Plan and design code by writing steps:
    - In English or another language. (Natural language)
    - In a diagram.
    - In **pseudocode** which is writing simplified code on paper.
- **Sequencing** defines the order for when steps in a process are completed.
    - Completed one at a time.

1.1.2 - First Java Program

- Every program in Java is written as a **class**.
    - Java is an **object-oriented language**, and **classes** output **objects** created from classes.
        - Basic building blocks in object-oriented programming (OOP).
    - Inside class, there is a **main method** that starts the program.
        - **Method** = block of code that performs a specific task.
        - Always the entry point of a program. → Java starts here on run-time.
- In java, must have open curly braces → used to start/end class definitions and method definitions.

1.1.3 - Compiling and Running Java Programs

- Written in any text editor including a small text editor built into Active Code exercises.
    - **Integrated Development Environment** (IDE) is used to write programs because you can compile, run, and write code.

        - .java extension in file names + must have inside a class in a source file, and it must match the file name.
- Computers **compile** (translate) Java into Java source files that it can understand.
    - A **compiler** checks code for errors and translates them into executable code that can be run.
    - Errors must be fixed before the program can run.
        - In Active Code, Java code is being sent to a server to be compiled, and errors/output are sent back to the webpage.

1.1.4 - Java Keywords

- **Keywords** are reserved words that have a special meaning in Java.
    - public, class, void are lowercase.
    - System and String are capitalized → both are classes.
- Lines in Java that express a complete action (such as printing output) must end with a semicolon. (;)
    - Such a line is a **statement**. (sort of like a period)
    - Won't be penalized on the exam if you forget a semicolon, but the Java compiler is not so lenient.
    - Not *every* line needs a semicolon, if the line starts a construct (if statement), then no semicolon is needed before opening.

1.1.5 - Syntax Errors and Debugging

- **Syntax errors** are reported from a compiler in Java if it is not written correctly.
    - Semicolon missing code has an open curly brace or open quote, but no closing one.
    - Informally a syntax error is a **bug**. The process of removing errors is **debugging**.
    - **Grace Hopper** documented a real bug that flew into a computer in 1947.
- If your code has **syntax errors** → error messages displayed below code.
    - Compile error messages tell you the type of error and what line.
    - All coders get bugs, **rubber duck debugging** is a common strategy.
        - Explain it to an inanimate object, you'll find your own mistakes.

1.1.6 - Reading Error Messages

- A **compile time error** happens when an error is detected by the compiler.
    - Much easier to find the problem if you know how to understand the error message.
    - The first line starts with the name of the file and class, then there is a colon followed by a number → line in which the problem was detected.
    - Line numbers may not be 100% accurate of what the problem within your program is.
    - After the line number and another colon, you will find the error message.
        - Can be cryptic but as you learn Java, become more apparent.
    - Next two lines depict your code and where the Java compiler thinks the error is, usually with a caret (^) symbol.

1.1.7 - Run-time Errors

- Some errors are not detected by the compiler → **run-time errors**.
    - Caused as the program is running, things such as dividing by zero, reading from a file that doesn't exist, or a logic error.
- An **exception** is a type of run-time error that results from unexpected behavior of the program that wasn't detected by the compiler.
    - Interrupts normal flow of program's execution → Java may report it.
    - Dividing by zero → **ArithmeticException** (report).
- Hard to catch as it doesn't cause the program to crash → behave in unexpected ways because of logic errors.
    - **Logic errors** are mistakes in an algorithm or program that makes it behave incorrectly or unexpectedly → run-time error.

1.1.8 - Comments

- Comments are with //, for multi-line comments use /* to start and */ to end.
    - Compiler skips over comments, but useful to make notes for you or another partner.

1.1.9 - Debugging Challenges

- Useful to work together in pairs for programming challenges.
    - Successful software development technique where two programmers work at 1 computer.
    - Driver types in code, navigator gives ideas and feedback.
    - Switch frequently.
    - If working alone → rubber duck debugging.

1.1.10 - Summary

- **Algorithms** define step-by-step processes to follow when completing a task or solving a problem. These algorithms can be represented using written language or diagrams.
- **Sequencing** defines an order for when steps in a process are completed. Steps in a process are completed one at a time.
- An **Integrated Development Environment (IDE)** is often used to write programs because it provides tools for a programmer to write, compile, and run code.
- A basic Java program looks like the following:

```java
public class MyClass
{
    public static void main(String[] args)
    {
        System.out.println("Hi there!");
    }
}
```

- A Java program starts with **public class NameOfClass {**. If you are using your own files for your code, each class should be in a separate file that matches the class name inside it, for example NameOfClass.java.
- Most Java classes have a main method that will be run automatically. It looks like this: **public static void main(String[] args) { }**.
- The **System.out.println()** method displays information given inside the parentheses on the computer monitor.
- Java statements end in ; (semicolon). { } are used to enclose blocks of code. // and /* */ are used for comments.
- A **compiler** translates Java code into a class file that can be run on your computer. **Syntax errors** are reported to you by the compiler if the Java code is not correctly written. Some things to check for are ; at the end of lines containing complete statements and matching { }, (), and "".
- A **syntax error** is a mistake in the program where the rules of the programming language are not followed. These errors are detected by the compiler.
- A **logic error** is a mistake in the algorithm or program that causes it to behave incorrectly or unexpectedly. These errors are detected by testing the program with specific data to see if it produces the expected outcome.
- A **run-time error** is a mistake in the program that occurs during the execution of a program. Run-time errors typically cause the program to terminate abnormally.
- An **exception** is a type of run-time error that occurs as a result of an unexpected error that was not detected by the compiler. It interrupts the normal flow of the program's execution.

1.1.11 - AP Practice

Java

```
System.out.println("Roses are red, ")              // Line 1;

System.out.println("Violets are blue, ")           // Line 2;

System.out.println("Unexpected '}' on line 32. ")  // Line 3;
```

- The code segment is intended to produce the following output but may not work as intended.

```
None

Roses are red,

Violets are blue,

Unexpected '}' on line 32.
```

- Which change, if any, can be made so that the code segment produces the intended output?
    - A. Replacing System with system on all lines.
    - B. Replacing println with print on lines 1 and 2.
    - C. Removing the single quotes in line 3.
    - **D. Putting the semicolon after the) on each line.**

1.2 - Variables and Data Types

1.2.1 - What is a Variable

- A **variable** is a memory location in the computer that stores a value, and can change or vary while a program is running.
    - Example: when playing a game, the score is a variable.
- Primitive types for AP CSA are:
    - int - integers such as 3, 0, -76, etc.
    - double - non-integer numbers like 6.3, -0.9, 602134.1234.
        - Called “floating point” because the decimal point “floats” relative to the magnitude of the number → scientific notation like 6.5×10^8.
        - Represented using 64 bits, doubling the type float.
            - Float not used for AP CSA.
    - boolean - represents two values: true and false. (named after George Boole, who invented boolean ALgebra)
    - string - An object type on the exam + name of a class in Java.
        - Enclosed in a pair of double quotation marks. (“Hello”)

- Data types have sets of values (a domain) and a set of operations on them. For instance, you can do addition and subtraction on ints and doubles, but not with booleans or strings.

1.2.2 - Declaring Variables in Java

- To declare a variable, you must tell Java its data type and name. Creating a variable = **declaring a variable**.
    - Type = keywords like int, double, or boolean.
    - Name = any name that you want, so long it follows convention.
- When creating a variable → Java makes a **primitive variable** with enough bits of memory and a location with the name you used.
- Computers store all values with **bits** (binary digits).
    - Bit represents 2 values: 0 and 1.
    - The three different primitive types have different numbers of bits.
        - Integer = 32 bits, double = 64 bits, Boolean = 1 bit
- To **declare** a variable → specify the type, leave a space, name the variable, and then place a semicolon.
    - You can also set the initial value right after declaration.
- When printing out variables, use the **string concatenation** (+) to add them to another string inside System.out.print.
    - Never put variables inside “” quotes → treated as a string.’
- Equal sign (=) does not mean equality in this context, but rather assignment.

1.2.3 - Naming Variables

- When naming variables, there are some rules that must be followed.
    - Cannot use any keyword or reserved words as variable names.
    - List of all keywords and reserved words.
    - The name of a variable should describe the data it holds.
        - Name like score → easy to read
        - x → not a good name, no clue what it's used for.
        - theNumberThatHoldsTheScore → too long/hard to read.

- Use **camel case**, it's important to note that variable names are case-sensitive and spelling sensitive.
  - camelCase → first letter not capitalized, all following first letters of words are capitalized.

1.2.4 - Debugging Challenge : Weather Report

```java
public class Challenge1_2_weather

{

   public static void main(String[] args)

   {

        double temperature = 70.5;

        int tvChannel = 101;

        boolean sunny = true;



        System.out.print("Welcome to the weather report on Channel ");

        System.out.println(tvChannel);

        System.out.print("The temperature today is ");

        System.out.println(temperature);

        System.out.print("Is it sunny today? ");

        System.out.println(sunny);

   }

}
```

1.2.5 - Coding Challenge : Mad Libs

Java

```java
public class MadLibs1

{

    public static void main(String[] args)

    {

        // fill these in with silly words/strings (don't read the poem yet)

        String pluralnoun1 = "cats";

        String color1 = "blue";

        String color2 = "red";

        String food = "tacos";

        String pluralnoun2 = "dogs";



        // Run to see the silly poem!

        System.out.println("Roses are " + color1);

        System.out.println(pluralnoun1 + " are " + color2);

        System.out.println("I like " + food);

        System.out.println("Do " + pluralnoun2 + " like them too?");



        // Now come up with your own silly poem!

        System.out.println("Roses are " + pluralnoun2);

        System.out.println(color2 + " are " + pluralnoun1);

        System.out.println("I like " + color1);
```

```
        System.out.println("Do " + food + " like them too?");
    }
}
```

1.2.6 - Summary

- (AP 1.2.B.2) A **variable** is a memory storage location that holds a value, which can change while the program is running.
- (AP 1.2.B.2) Every variable has a name and an associated data type that determines the kind of data it can hold. A variable of a primitive type holds a primitive value from that type.
- A variable can be declared and initialized with the following code:

```java
int score;

double gpa = 3.5;
```

- (AP 1.2.A.1) A **data type** is a set of values and a corresponding set of operations on those values. Data types can be primitive types (like int) or reference types (like String).
- (AP 1.2.A.2) The **primitive** data types used in this course define the set of values and corresponding operations on those values for numbers and Boolean values.
- (AP 1.2.A.3) A **reference** type, like String, is used to define objects that are not primitive types.
- (AP 1.2.B.1) The three primitive data types used in this course are **int** (integer numbers), **double** (decimal numbers), and **boolean** (true or false).
- String is a reference data type representing a sequence of characters.

1.2.7 - AP Practice

- Which of the following pairs of declarations are the most appropriate to store a student's average course grade in the variable GPA and the number of students in the variable numStudents?
    - A. int GPA; int numStudents;
    - **B. double GPA; int numStudents;**
    - C. double GPA; double numStudents;
    - D. int GPA; boolean numStudents;
    - E. double GPA; boolean numStudents;

1.3 - Expressions and Output

1.3.1 - Output

- Two main methods of outputting:
    - **System.out.println(value)**: prints value followed by a new line (ln).
    - **System.out.print(value)**: prints value without advancing to a new line.
- Anything within quotation marks is denoted as a **string literal**, which is zero to many characters enclosed and ending double quotes.
    - A **literal** is the code representation of a fixed value.
        - Numerical or string.
- If we want to print special characters, such as double-quotes, place a backslash before the character.
    - Called a **backlash escape sequence** → you can backlash the backlash as well.
    - \n itself will also print another new line.

1.3.2 - Expressions and Operators

- **Arithmetic Expressions** consist of numerical values, variables, and arithmetic operators.
    - Addition (+), subtraction (-), division (/), multiplication (*).
    - + can also be used to concatenate.
- Arithmetic expressions can be of int or double.
    - Expressions with only int values evaluate to an int value.

    - Any arithmetic expression that has at least one double value will evaluate to a double value.
        - When doing division with two integers, you will get the integer result but not the decimal → **truncating division**.\

1.3.3 - Compound Expressions

- Expressions can be combined into **compound expressions** that have multiple operators. When compound expressions are evaluated, **operator precedence** is used.
    - PEMDAS → Parentheses are often used in complex expressions.

1.3.4 - The Remainder Operator

- % operator is the **remainder** operator. It takes two operands and returns the remainder after dividing the first from the second.
    - Truncating integer division.
- Sort of like the remainder number after you do a long division.
- % modulo called **modulo** or **mod**.
    - Actually different as modulo uses Euclidean division.
        - Only matters when you have negative operands.
        - Need to know % for AP Exam only.

1.3.5 - Coding Challenge : Pay Calculator

```
Java

public class Challenge1_3_Pay_Calculator

{

   public static void main(String[] args)

   {

       // Put in the math operator between 4 and 10 below to compute
```

```
        // pay for 4 hours of work at 10 dollars per hour.
        System.out.println("Pay for 4 hours of work at 10 dollars an hour");
        System.out.println(4  * 10);

        // Put in the math operator to compute the number of hours worked
        // if the pay is 120 dollars and the rate is 15 dollars per hour.
        System.out.println("Number of hours worked for pay 120 dollars & rate 15 dollars per hour");
        System.out.println(120 / 15);

        // Put in the math expression to compute the pay
        //  for 12 hours of work at 7.50 dollars per hour.
        System.out.println("Pay for 12 hours of work at 7.50 dollars an hour");
        System.out.println(12 * 7.50);

        // Add another statement to print the math expression to compute the integer number of
        // hours worked if the pay is 100 dollars and the rate is 9 dollars per hour.
        System.out.println("Number of int hours worked for pay 100 dollars & rate 9 dollars per hour");
        System.out.println(100 / 9);
```

```
        // Add another statement to print the math expression to give the remainder
when
        //  100 dollars is divided by 9 dollars per hour.
        System.out.println("The remainder of 100 dollars divided by 9 dollars per
hour");
        System.out.println(100 % 9);


    }
}
```

1.3.6 - Summary

- (AP 1.3.A.1) System.out.print and System.out.println are Java output statements that display information on the computer screen. System.out.println moves the cursor to a new line after the information has been displayed, while System.out.print does not.
- (AP 1.3.B.1) A **literal** is the code representation of a fixed value, which can be a string or a numerical value.
- (AP 1.3.B.2) A **string literal** is a sequence of zero to many characters enclosed in starting and ending double quotes.
- (AP 1.3.B.3) **Escape sequences** are special sequences of characters that can be included in a string. They start with a ` and have a special meaning in Java. Escape sequences used in this course include double quote `" , backslash `, and newline `n.
- (AP 1.3.C.1) **Arithmetic expressions**, which consist of numeric values, variables, and operators, include expressions of type int and double.
- (AP 1.3.C.2) The arithmetic **operators** consist of +, -, * , /, and % also known as addition, subtraction, multiplication, division, and remainder.

- (AP 1.3.C.2) An arithmetic operation that uses two int values will evaluate to an int value. With integer division, any decimal part in the result will be thrown away.
- (AP 1.3.C.2) An arithmetic operation that uses at least one double value will evaluate to a double value.
- (AP 1.3.C.3) When dividing numeric values that are both int values, the result is only the integer portion of the quotient. Anything after the decimal point is thrown away. When dividing numeric values that use at least one double value, the result is the double quotient as expected.
- (AP 1.3.C.4) The **remainder (modulo) operator %** is used to compute the remainder when one number is divided by another number.
- (AP 1.3.C.5) Multiple operators can be used to combine expressions into **compound expressions**.
- (AP 1.3.C.5) During evaluation, numeric values are associated with operators according to **operator precedence** to determine how they are grouped. *, /, % have precedence over + and -, unless parentheses are used to group those to be evaluated first. Operators with the same precedence are evaluated from left to right.
- (AP 1.3.C.6) An attempt to divide an integer by zero will result in an ArithmeticException.

1.3.7 - AP Practice

```
Java

System.out.print("Java is ");

System.out.println("fun ");

System.out.print("and cool!");
```

- What is printed as a result of executing the code segment?
    - A. “Java is fun and cool!”
    - B. “Java isfun \n and cool!”

- C. “Java is \n fun \n and cool!”
- **D. “Java is fun \n and cool!**

1.4 - Assignment and Input

1.4.1 - Assignment Statements

- **Assignment statements** initialize/change values stored in a variable using the assignment operator (=).
  - The assignment statement always has a single variable on the left side.
  - Value of **expression** on the right side.
- Can set the value of one variable to another one; copy the variable.

1.4.2 - Data Types in Assignments

- Every variable must be assigned a value before it can be used in an expression.
  - Must be a compatible data type.
  - A variable is **initialized** the first time you assign it a value.
  - Value type is based on the type used in expressions.
    - If you have at least one double value → expression is double.
    - Only int values → expression is int.
- Reference types like String can be assigned a new object/null if there is no object.
  - Literal **null** is a special value → isn’t associated with any object.

1.4.3 - Adding 1 to a Variable

- If we use variables, we are likely to increment them.
  - Do this through variable = variable + 1.
  - Can use the value of a variable within reassignment.

1.4.4 - Input with Variables

- Input is anything the user enters into the program, many types of input.
  - **Tactile** through buttons, **audio** with speech, **visual** with webcam, or most commonly, **text** that user types in.
  - Scanner class obtains text input from keyboard.

- Declare scanner with import java.util.Scanner;
- Newer Console class that has more capabilities.
    - Use IDEs and other coding environments for input/output.

1.4.5 - Coding Challenge : Dog Years

```java
public class Challenge1_4
{
    public static void main(String[] args)
    {
        // Fill in values for these variables
        int currentYear = 2025;
        int birthYear = 2009;
        int dogBirthYear = 2019;

        // Write a formula to calculate your age from the currentYear and
        // your birthYear variables
        int age = currentYear - birthYear;

        // Write a formula to calculate your dog's age from the currentYear
        // and dogBirthYear variables
        int dogAge = currentYear - dogBirthYear;

        // Calculate the age of your dog in dogYears (7 times your dog's age
```

```
        // in human years)
        int dogYearsAge = dogAge * 7;


        // Print out your age, your dog's age, and your dog's age in dog
        // years. Make sure you print out text too so that the user knows what
        // is being printed out.
        System.out.println("Current Year: " + currentYear + "\nBirth Year: " +
birthYear);
        System.out.println("Dog Birth Year: " + dogBirthYear + "\nYour Age: " +
age);
        System.out.println("Dog Age: " + dogAge + "\nDog Age In Dog Years: " +
dogYearsAge);


   }
}
```

1.4.6 - Summary

- (AP 1.4.A.2) The **assignment operator** (=) allows a program to initialize or change the value stored in a variable. The value of the expression on the right is stored in the variable on the left.
- (AP 1.4.A.1) Every variable must be assigned a value before it can be used in an expression. That value must be from a compatible data type.
- (AP 1.4.A.1) A variable is **initialized** the first time it is assigned a value.
- (AP 1.4.A.1) Reference types can be assigned a new object or null if there is no object. The literal null is a special value used to indicate that a reference is not associated with any object.

- (AP 1.4.A.3) During execution, an expression is evaluated to produce a single value. The value of an expression has a type based on the types of the values and operators used in the expression.
- (AP 1.4.B.1) Input can come in a variety of forms, such as tactile, audio, visual, or text. The Scanner class is one way to obtain text input from the keyboard, although input from the keyboard will not be on the AP exam.

1.4.7 - AP Practice

```java
int a = 5;

int b = a/2;

double c = a/2.0;

double d = 5 + a / b * c - 2;

System.out.println(d);
```

- What is printed when the code segment is executed?
    - A. 8
    - **B. 8.0**
    - C. 10.5
    - D. An incompatible type error will occur.

1.5 - Casting and Range Values

1.5.1 - Casting

- **Type casting** converts values from one type → another type.
    - **Casting** refers to "reshaping" data through a **cast** operator.
    - The **cast operator** looks like (int) or (double) before any expression.
    - For example, (double) 1 / 3 → 0.33333 instead of a truncated 0.
    - Casting an int value to a double adds digits to the right of the decimal point.

    - In the opposite way, you are truncating the value if switching from double to int.
- Division of two integers in Java → int result with a truncated (no decimal/round down) value.
    - 9 / 10 → 0, not 1!
- Any expression containing a double will cause the entire thing to also be a double; it is contagious.
    - 9.0 / 10 → 0.9
    - Conversion from int to double is called a **widening conversion** → any int value can be represented by a double, but not vice versa.
    - Note: ints are 32-bit signed (positive and negative) values that go from -2^{31} to $2^{31} - 1$. Doubles can represent -2^{53} to 2^{53}, and also much larger values but limited precision.
    - Integer.MIN_VALUE and Integer.MAX_VALUE give these values to you.

1.5.2 – Rounding

- From double to int, we round down (truncation) to the nearest integer.
    - 7.0 / 4.0 = 1.75 → 1 if converted to an integer.

1.5.3 - Range of Values

- Java's doubles store 14-15 digits worth of precision.
- Int values are stored in 4 bytes of memory; the largest int value is 2147483647 while the smallest is -2147483648. If you try to store any number larger or smaller, it results in an **integer overflow** where an incorrect value could be stored.
- Computers dedicate a specified amount of memory to store data for each data type.
    - Double values have twice as much memory than for ints.
    - To avoid round-off errors, round the result to a specified number of digits after the decimal point or to an int value.
- Although not on the AP Exam, you can format long decimal numbers to show 2 digits after the decimal point using printf instead of println. %.02f tells printf to print a floating number indicated by the % with 2 digits after the decimal point.

1.5.4 - Coding Challenge : Average 3 Numbers

Java

```java
public class Challenge1_6
{
    public static void main(String[] args)
    {
        // 1. Declare 3 int variables called grade1, grade2, grade3
        // and initialize them to 3 values
        int grade1 = 90, grade2 = 75, grade3 = 67;

        // 2. Declare an int variable called sum for the sum of the grades
        int sum;

        // 3. Declare a variable called average for the average of the grades
        double average;

        // 4. Write a formula to calculate the sum of the 3 grades (add them
        // up).
        sum = grade1 + grade2 + grade3;

        // 5. Write a formula to calculate the average of the 3 grades from
        // the sum using division and type casting.
        average = (double) (sum) / 3;
```

```
        // 6. Print out the average
        System.out.println("average = " + average);


    }
}
```

1.5.5 - Bonus Challenge : Unicode

- Java was one of the first programming languages to use UNICODE instead of ASCII.
- The Unicode people declared they would only need 65536 to represent all characters used in the world → Java included a char type that can hold exactly 2^{16} values.
- Unicode actually has 1,112,064 possible codepoints → made char obsolete.
- You can use a string to represent a codepoint and the method Character.toString to translate an int → String containing the character for that codepoint.

```
Java
public class ChallengeUnicode
{
    public static void main(String[] args)
    {
        System.out.println(
                "'A' in ASCII and Unicode: " + Character.toString(65));
        System.out.println("Chinese for 'sun': " + Character.toString(11932));
        System.out.println("A smiley emoji: " + Character.toString(128512));
```

```
        // Old style. Doesn't work for all codepoints.
        System.out.println("This also works: " + (char) 65);
        System.out.println("But this doesn't: " + (char) 128512);

        System.out.println(Character.toString(7531));
        System.out.println(Character.toString(120196));
        System.out.println(Character.toString(120456));
    }
}
```

1.5.6 - Summary

- **Type casting** is used to convert a value from one type to another.
- (AP 1.5.A.1) The casting operators (int) and (double) can be used to convert from a double value to an int value (or vice versa).
- (AP 1.5.A.2) Casting a double value to an int causes the digits to the right of the decimal point to be truncated (cut off and thrown away).
- (AP 1.5.A.3) Some code causes int values to be automatically cast (widened) to double values. In expressions involving doubles, the double values are contagious, causing ints in the expression to be automatically converted ("widened") to the equivalent double value so the result of the expression can be computed as a double.
- (AP 1.5.A.4) Values of type double can be rounded to the nearest integer by (int)(x + 0.5) or (int)(x – 0.5) for negative numbers.
- (AP 1.5.B.1) The constant Integer.MAX_VALUE holds the value of the largest possible int value. The constant Integer.MIN_VALUE holds the value of the smallest possible int value.

- (AP 1.5.B.2) Integer values in Java are represented by values of type int, which are stored using a finite amount (4 bytes) of memory. Therefore, an int value must be in the range from Integer.MIN_VALUE to Integer.MAX_VALUE, inclusive.
- (AP 1.5.B.3) If an expression evaluates to an int value outside of the allowed range, an **integer overflow** occurs. The result is an int value in the allowed range but not necessarily the value expected.
- (AP 1.5.C.1) Computers allot a specified amount of memory to store data based on the data type. If an expression evaluates to a double that is more precise than can be stored in the allotted amount of memory, **a round-off** error occurs. The result will be rounded to the representable value. To avoid rounding errors that naturally occur, use int values or round doubles to the precision needed.

1.5.7 - AP Practice

- Which of the following returns the correct average for a total that is the sum of 3 int values?
    - A. (double) (total / 3);
    - B. total / 3
    - C. (double) total / 3;
    - D. (int) total / 3;

1.6 - Compound Assignment Operators

- Compound assignment operators += , -=, *=, /=, and %= are shortcuts that do math operation and assignment in one step.
    - x += 1 is the same as x = x + 1
- Since some values are especially common, two even more concise operators ++ and – (also called plus-plus or **increment** operator, similarly minus-minus or **decrement** operator).
- If the ++ operator is before the variable name (called the **postfix** operator), the value of the variable is changed after evaluating the variable to get its value.
    - If it is before the variable, the value of the variable is incremented before the variable is evaluated to get the value of the expression.

1.6.1 - Code Tracing Challenge and Operators Maze

- **Code Tracing** is a technique used to simulate a dry run through code or pseudocode line by line by hand as if you are the computer executing the code.
    - Used for debugging or proving that a program runs correctly.

1.6.2 - Summary

- (AP 1.6.A.1) Compound assignment operators (+=, -=, *=, /=, %=) can be used in place of the assignment operator in numeric expressions. A compound assignment operator performs the indicated arithmetic operation between the current value of the variable on the left and the value on the right and then assigns the resulting value to the variable on the left.
- (AP 1.6.A.2) The increment operator (++) and decrement operator (--) are used to add 1 or subtract 1 from the stored value of a numeric variable. The new value is assigned to the variable.

1.7 - APIs and Libraries

- When we used code like System.out.println(“hello, world!”) to print to the screen, we didn’t really know what it meant.
    - Namely, it causes the text “hello, world!” to appear on the screen.
- A collection of code written by other programs is called a **library** and the details of how to use them are described in its **Application Programming Interface**, or **API** for short. The programming line above is a part of the Java API, a large set of code that comes with Java.
    - Essential to programming because they allow us to focus on the specific task we are trying to accomplish without having to rewrite the same bits of functionality over and over.

1.7.1 - The Turtle Library: Classes, Attributes, and Behaviors

- Turtle Java library allows us to create drawings using animated turtles that move around the screen.
- Defined after a **class**, a **class** has specific reference types used as building blocks for object-oriented programming.

    - Each class has a new kind of value similar to how each primitive type like int or double defines a particular kind of value.
    - These values that are defined by classes are called **objects** → where **object-oriented programming** gets its name.
- Classes define the structure of objects in terms of **attributes** and **behaviors**.
    - **Attributes** - data related to the class and stored in variables.
    - **Behaviors -** what instances of the class can do; defined by methods.
- Example attributes every Turtle object has are color, xPos, yPos.
- Behaviors implemented onto Turtle objects include forward(), backward(), turnLeft(), and turnRight().
- Turtle objects have a name, and after creating the object, you use a **dot operator** to access the attributes and methods of the object.
- **Documentation** found in API specifications allows us to understand the attributes and behaviors of a class.

1.7.2 - Packages

- Classes are the main building blocks of Java programs, but libraries and programs are made up of multiple classes that are used together.
- Related classes are grouped into a **package**, sort of like folders where we store related documents.
- When we want to use a class defined in a package → we **import** it and use it as if we wrote the program.
- The java.lang package contains key classes such as String and System, which are documented in **Javadocs**, or Java's online **API documentation**.
    - Contains all publicly available attributes (called **fields** in Javadocs) and behaviors (called **methods**).

1.7.3 - Coding Challenge: Turtle Drawing

```
Java

import java.awt.*;
```

```
import java.util.*;


public class TurtleShape
{
    public static void main(String[] args)
    {
        World habitat = new World(500, 500);
        Turtle yertle = new Turtle(habitat);


        // Use any of yertle's methods such as forward, turnRight, and turnLeft
        // to draw a shape
        yertle.forward();
        yertle.turnRight();
        yertle.turnLeft();
        yertle.forward();
        yertle.turnRight();
        yertle.turnLeft();
        yertle.backward();
        yertle.turnLeft();
        yertle.forward();


        // Do not change the line below!
        habitat.show(true);
```

```
    }
}
```

1.7.4 - Summary

- (AP 1.7.A.1) Libraries are collections of classes written by other programmers.
- (AP 1.7.A.1) An Application Programming Interface (API) specification informs the programmer how to use classes in a library.
- (AP 1.7.A.1) Documentation found in API specifications and libraries is essential to understanding the attributes and behaviors of a class defined by the API.
- (AP 1.7.A.1) Classes in the APIs and libraries are grouped into packages that can be imported into a program.
- (AP 1.7.A.1) A class defines a specific reference type and is the building block of object-oriented programming. Existing classes and class libraries can be utilized to create objects.
- (AP 1.7.A.2) Attributes refer to the data related to the class and are stored in variables.
- (AP 1.7.A.2) Behaviors refer to what instances of the class can do (or what can be done with them) and are defined by methods.

1.7.5 - AP Practice

- The following is an excerpt from a class specification for the API for a Turtle class.
    - An int variable called xPos to represent the x-coordinate of the turtle
    - An int variable called yPos to represent the y-coordinate of the turtle
    - A method called forward() that moves the turtle forward by 100 pixels
    - A method called turnRight() that turns the turtle 90 degrees to the right
- Based on the Turtle API above, which of the following descriptions is accurate?
    - A. The xPos and yPos are behaviors of Turtle objects.
    - **B. The xPos and yPos are attributes of Turtle objects.**
    - C. forward is an attribute of Turtle objects.
    - D. Turtle is an attribute of the API.

- The following is an excerpt from a class specification for the API for a Turtle class.
    - An int variable called xPos to represent the x-coordinate of the turtle
    - An int variable called yPos to represent the y-coordinate of the turtle
    - A method called forward() that moves the turtle forward by 100 pixels
    - A method called turnRight() that turns the turtle 90 degrees to the right
- Based on the Turtle API above, which of the following descriptions is accurate?
    - A. Turtle is an attribute of the API.
    - B. The forward method is an attribute of Turtle objects.
    - **C. The turnRight method is a behavior of Turtle objects.**
    - D. xPos is a behavior of Turtle objects.

1.8 - Documentation with Comments and Preconditions

1.8.1 - Comments

- Comments make code more readable + maintainable.
    - Essential in this kind of environment + good habit to develop.
    - Written for both the original programmer + other programmers to understand.
    - Ignored by compiler + not runned by Java.
- Three types of comments:
    - // - Single line comment
    - /* Multiline block of comments */
    - /** Java documentation comments */
        - A special version of multi-line, javadoc comes with Java JDK that will put these comments to make documentation of a class a web page.
        - Generates official Java documentation like the String class.
        - Not used in AP Exam, but a good idea to use it for classes, methods, and instance variables if you want to save it as a library.'
- Common tags used in commenting include:
    - @author Author of the program
    - @since Date released
    - @version Version of program
    - @param Parameter of a method

- @return Return value for a method

1.8.2 - Preconditions and Postconditions

- Methods in API libraries have **preconditions** and **postconditions** described in their comments.
  - **Precondition** - condition that must be true for a method code to work. It is what the method expects in order to do its job properly.
    - Dividing by zero or computing square root of negative numbers, etc.
  - **Postcondition** - condition that is true after running the method → describe the outcome of running a method. Useful as you can check and see if you got the correct results

1.8.3 - Software Validity and Use-Case Diagrams

- Not covered by AP Exam, but useful for professional programmers.
- Determining preconditions/postconditions of our code helps determine the **validity** of our software.
  - If the code is part of a satellite → going to outer space, we want to test it thoroughly and make sure it works and is valid before put into use.
  - Break your code! Try all kinds of inputs.
- Software designers draw a high-level **Use-Case Diagram** of a system that shows different ways a user might interact with a system before they build it.
  - A **Use-Case** is an user interaction or situation in the system/software.
  - Designers write down the preconditions and postconditions of each Use-Case.

1.8.4 - Agile Software Development

- Many different models for software development → **waterfall method** developed in the 1970s.
  - Criticized since not adaptable → **Agile** development model.
    - Teams work in short 2-3 week sprints to develop/test/release a component of the project.

1.8.5 - Coding Challenge: Preconditions in Algorithms

```
None

Preconditions for "Make Account": Customer is an adult. The system is up and
running.

Postconditions for "Make Account": Customer searches products. The system has
available products.

Preconditions for "Buy Book": Customer has account and method of payment. The
system has an available product.

Postconditions for "Buy Book": Customer orders book. The system places an order to
manufacture and ship.

Preconditions for "Manufacturing": Customer ordered book. The system has available
workers.

Postconditions for "Manufacturing": Customer receives a final date. The system
ships the book.

Preconditions for "Shipping": Customer has a date and address. The system has a
manufactured book.

Postconditions for "Shipping": Customer receives book. System completes purchase,
```

1.8.6 - Summary

- (AP 1.8.A.1) **Comments** are written for both the original programmer and other programmers to understand the code and its functionality, but are ignored by the compiler and are not executed when the program is run.
- (AP 1.8.A.1) Three types of comments in Java include /* */, which generates a block of comments, //, which generates a comment on one line, and /** */, which are Javadoc comments and are used to create API documentation.
- (AP 1.8.A.2) A **precondition** is a condition that must be true just prior to the execution of a section of program code in order for the method to behave as expected. There is no expectation that the method will check to ensure preconditions are satisfied.
- (AP 1.8.A.3) A **postcondition** is a condition that must always be true after the execution of a section of program code. Postconditions describe the outcome of the

execution in terms of what is being returned or the current value of the attributes of an object.

1.8.7 - AP Practice

```
Java

        /** method to add extra-credit to the score **/

        public double computeScore(double score, double extraCredit)

        {

            double totalScore = score + extraCredit;

            return totalScore;

        }


Which of the following preconditions are reasonable for the computeScore method?
```

- A. /* Precondition: score <= 0 */
- **B. /* Precondition: score >= 0 */**
- **C. /* Precondition: extraCredit >= 0 */**
- D. /* Precondition: extraCredit <= 0 */
- E. /* Precondition: computeScore >= 0 */

1.9 - Method Signatures

1.9.1 - Methods and Procedural Abstraction

- All of our code has been in the main method, but complex programs are made up of many methods.
    - Large problems → break into smaller subproblems and solve them separately.
- A **method** is a named block of code that performs a task when it is called. (A **block** of code is any section of code enclosed in curly braces.)
    - These named blocks go by many names: functions, procedures, subroutines, etc.

- **Procedural abstraction** allows a programmer to not worry about the details of a method → just know that it works and the method's name.

1.9.2 - Method Calls

- Divide a program into multiple methods to organize the code and reduce complexity; avoid repetition of code.
- A **method** call is when the code "calls out" the method's name in order to run the code in the method. They always include parentheses () and sometimes some data inside the parentheses if the method needs it to do its job.
    - The **flow of control** skips from method to method as they are called. Once it is done using the method, it returns back to the line right after the method call.
        - Figuring out the flow of control is called **tracing** through the code.

1.9.3 - Methods Signature, Parameters, Arguments

- When using methods in a library or API, look up the **method signature** (or **method header**) in its documentation.
    - **Method header** - method header without return type, just the method and its parameter list. It is the first line of a method that includes its name, return type, and the parameter list of parameters and their data types.
    - **Return type** - type of value that a method returns.
    - **Void** - method doesn't return anything.
- For values that are passed onto methods, we call them an **argument**, or the actual value that is passed to the method.
- Make methods even more powerful by giving them parameters for data that they need to do their job. A **parameter** (or **formal parameter**) is a variable declared in the header of a method or constructor → used inside the body of the method.
    - Allows values or arguments to be used by the method.
        - Again, an **argument** (called an **actual parameter**) is a value that is passed into the method.
- If a method is called → the right method definition is found by checking the **method signature** or **header** at the top of the method definition. A method signature for a

method with parameters consists of the method name and the ordered list of parameter types.
    - Contain data types because they are variables that reverse memory for parameter variables.
- Java uses **call by value** when it passes arguments to methods. This means that a copy of the argument is made onto the parameter, meaning that if you change the parameter inside the method, the original value in your argument will not be changed.

1.9.4 - Overloading

- Methods are **overloaded** when there are multiple methods with the same name but different signatures or different numbers or types of parameters.

1.9.5 - Coding Challenge: Song with Parameters

```
None

The ants go marching one by one, hurrah, hurrah

The ants go marching one by one, hurrah, hurrah

The ants go marching one by one

The little one stops to suck a thumb

And they all go marching down to the ground

To get out of the rain, BOOM! BOOM! BOOM! BOOM!


The ants go marching two by two, hurrah, hurrah

The ants go marching two by two, hurrah, hurrah

The ants go marching two by two

The little one stops to tie a shoe

And they all go marching down to the ground

To get out of the rain, BOOM! BOOM! BOOM! BOOM!
```

```
The ants go marching three by three, hurrah, hurrah

The ants go marching three by three, hurrah, hurrah

The ants go marching three by three

The little one stops to climb a tree

And they all go marching down to the ground
To get out of the rain, BOOM! BOOM! BOOM! BOOM!
```

1.9.6 - Summary

- (AP 1.9.A.1) A **method** is a named block of code that only runs when it is called. A block of code is any section of code that is enclosed in braces.
- (AP 1.9.A.1) **Procedural abstraction** allows a programmer to use a method by knowing what the method does even if they do not know how the method was written.
- (AP 1.9.B.5) A method call interrupts the sequential execution of statements, causing the program to first execute the statements in the method before continuing. Once the last statement in the method has been executed or a return statement is executed, the flow of control is returned to the point immediately following where the method was called.
- (AP 1.9.A.2) A **parameter** is a variable declared in the header of a method or constructor and can be used inside the body of the method. This allows values or arguments to be passed and used by a method or constructor.
- (AP 1.9.A.2) A **method signature** for a method with parameters consists of the method name and the ordered list of parameter types. A method signature for a method without parameters consists of the method name and an empty parameter list.
- (AP 1.9.B.3) An **argument** is a value that is passed into a method when the method is called. The arguments passed to a method must be compatible in number and

order with the types identified in the parameter list of the method signature. When calling methods, arguments are passed using call by value.

- (AP 1.9.B.3) **Call by value** initializes the parameters with copies of the arguments.
- (AP 1.9.B.4) Methods are said to be **overloaded** when there are multiple methods with the same name but different signatures.

1.9.7 - AP Practice

Java

```
public class Cat

{

    public static void sound1()

    {

        System.out.print("meow ");

    }



    public static void sound2()

    {

        System.out.print("purr ");

    }



    public static void hello()

    {

        sound2();

        sound1();

    }

}
```

- Which of the following code segments, if located in a method inside the Cat class, will cause the message “purr meow purr” to be printed?
    - A. hello();
    - **B. hello(); sound2();**
    - C. sound1(); sound2(); sound1();
    - D. purr(); meow(); purr();

1.10 - Calling Class Methods

- Most methods we have used are **static methods**, or **class methods**.
    - Associated with a class and include the keyword static in the method header.
    - The main method is always static → only one copy of the method.
- Template for a static method:
    - In the **method header**, the keyword **static** is used before the **return type**.
    - We use the keyword **void** as the return type for methods that do not return a value.

1.10.1 - Non-void Methods

- Most methods we have used are **void methods**, which do not return a value.
- Many methods act as mathematical functions → return a value.
    - Called **non-void methods**; like a calculating machine that returns numbers.
- There could be an **overloaded** version of methods that take in a double number and return a double value.
    - Remember that methods are overloaded when there are multiple methods with the same name but different signatures with a different number/type of parameter.
- The **return** statement is used to return a value from a method back to the calling code.
    - Also ends the method execution, so any code after the return statement is not executed.

1.10.2 - Common Errors with Methods

- A common error is to forget to do something with the value returned from a method.
    - When you call a method that returns a value, you should do something with it such as assigning it to a variable or printing it out.
    - To use the return value, it should be stored in a variable as part of an expression.
- Another error is to mismatch the types or order for the arguments or return values.

1.10.3 - Methods Outside the Class

- If we want to call methods from a different class imported from libraries, you have to not only include the method name, but also the class name.
- Common methods (such as square) come from the MathFunctions class.
- Math.sqrt() → square root.
- Math.pow() → to the power of.

1.10.4 - Coding Challenge: Ladder on Tower

- Which expression would correctly compute the hypotenuse of a triangle?
- Math.sqrt(a * a + b * b) and Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));

1.10.5 - Summary

- (AP 1.10.A.1) Class methods are associated with the class (not instances of the class which we will see in later lessons).
- (AP 1.10.A.2) Class methods include the keyword **static** in the header before the method name.
- (AP 1.9.B.1) A void method does not have a return value and is therefore not called as part of an expression.
- (AP 1.9.B.2) A **non-void method** returns a value that is the same type as the **return type** in the header.
- (AP 1.9.B.2) To use the return value when calling a non-void method, it must be stored in a variable or used as part of an expression.
- Common errors with methods are mismatches in the order or type of arguments, return values, and forgetting to do something with the value returned from a

method. When you call a method that returns a value, you should do something with that value like assigning it to a variable or printing it out.

- (AP 1.10.A.2) Class methods are typically called using the class name along with the dot operator. When the method call occurs in the defining class, the use of the class name is optional in the call.

1.10.6 - AP Practice

```java
public static double calculatePizzaBoxes(int numOfPeople, double slicesPerBox)

{ /*implementation not shown */}
```

- Which of the following lines of code, if located in a method in the same class as calculatePizzaBoxes, will compile without an error?
    - A. int result = calculatePizzaBoxes(45, 9.0);
    - B. double result = calculatePizzaBoxes(45.0, 9.0);
    - C. int result = calculatePizzaBoxes(45.0, 9);
    - **D. double result = calculatePizzaBoxes(45, 9.0);**
    - E. result = calculatePizzaBoxes(45, 9);

```java
public void static inchesToCentimeters(double i)

{

    double c = i * 2.54;

    printInCentimeters(i, c);

}



public void static printInCentimeters(double inches, double centimeters)

{
```

```
    System.out.print(inches + "-->" + centimeters);
}
```

- Assume that the method called inchesToCentimeters(10) appears in a static method in the same class. What is printed as a result of the method call?
    - A. inches –> centimeters
    - B. 10 –> 25
    - C. 25.4 –> 10
    - D. 10 –> 12.54
    - **E. 10.0 –> 25.4**

1.11 - Using the Math Class

- Random numbers can be used with the Math.random() method -> generates a random number.
    - Lots of similar mathematical methods, such as Math.pow()
    - All can be found as part of the java.lang package → available by default.
        - All of these are **class methods** (or **static** methods) → call them by just using ClassName.methodName();

1.11.1 - Mathematical Functions

- These following methods are from the Math class that are required to know on the AP exam:
    - int abs(int): Returns the absolute value of an int value.
    - double ans(double): Returns the absolute value of a double value.
    - double pow(double, double): Returns the value of the first parameter raised to the power of the second parameter.
    - double sqrt(double): Returns the positive square root of a double value.
    - Double random(): Returns a double value greater than or equal to 0.0 and less than 1.0 (not including 1.0!).

- All of them are mathematical functions that compute new values based on arguments → you can intuitively guess what abs, pow, and sqrt do based on their abbreviations.
    - Since these are literal values →able to use them in print or assignment statements.

1.11.2 - Random Numbers

- Math.random() returns a double number greater than or equal to 0.0, less than 1.0
    - We have to be precise about the ranges (where it ends, and what is part of the range).
        - For instance, it can actually exactly return 0, but not exactly return 1.
        - Ranges have to be specified with inclusive/exclusive; 0 inclusive, 1 exclusive.
        - Many ranges in Java are expressed this way → inclusive bottom/exclusive top.
    - Though it may not seem useful, can **multiply** the random values to get larger ranges → stretch it to whatever range.
    - If you want to extend the inclusive/exclusive range → add/subtract numbers.

1.11.3 - Coding Challenge : Random Numbers

```
Java

public class MathChallenge

{

    public static void main(String[] args)

    {

        // 1. Use Math.random() to generate 3 integers from 0-40 (not

        // including 40) and print them out.

        int num1 = (int) (Math.random() * 40);

        int num2 = (int) (Math.random() * 40);
```

```
        int num3 = (int) (Math.random() * 40);


        // 2. Calculate the number of combinations to choose 3 numbers between
        // 0-40 (not including 40) using Math.pow() and print it out.
        double combinations = Math.pow(40.0, 3.0);
        System.out.println("number 1: " + num1 + "\nnumber 2: " + num2 + "\nnumber
3: " + num3);
        System.out.println("combinations: " + combinations);
        // For example, Math.pow(10,2) is 10^2 and the number of permutations
        // to choose 2 numbers between 0-9.


    }
}
```

1.11.4 - Summary

- (AP 1.11.A.1) The Math class is part of the java.lang package. Classes in the java.lang package are available by default.
- (AP 1.11.A.2) The Math class contains only class (static) methods. They can be called using **Math**.method(); for each method.
- (AP 1.11.A.2) The following static Math methods are part of the Java Quick Reference:
    - **int abs(int)** : Returns the absolute value of an int value (which means no negatives).
    - **double abs(double)** : Returns the absolute value of a double value.
    - **double pow(double, double)** : Returns the value of the first parameter raised to the power of the second parameter.
    - **double sqrt(double)** : Returns the positive square root of a double value.

    - **double random()** : Returns a double value greater than or equal to 0.0 and less than 1.0 (not including 1.0)!
- (AP 1.11.A.3) The values returned from Math.random() can be manipulated to produce a random int or double in a defined range.
- **(int)(Math.random() * range) + min** moves the random number into a range starting from a minimum number. The range is the **(max - min + 1)**. For example, to get a number in the range of 5 to 10, use the range 10-5+1 = 6 and the min number 5: (int)(Math.random()*6) + 5.

1.12 - Objects - Instances of Classes

- Java is **object-oriented programming** language → 1 of primary ways of designing + organizing a Java program is through **objects**.
    - Combine data and code that operates on that data into 1 unit.
    - First define a **class** to create an object → blueprint for said objects.
    - All java programs made from classes → explanation as to why they start with public class.

1.12.1 - What are Classes and Objects?

- Class → blueprint of a house
- Houses → objects of said blueprint.
- Classes can also be thought of defining a new data type → just like int variables are used to declare variables that hold numbers, the Turtle can also be used to declare many animated turtle objects, which are **instances** of the Turtle class.

1.12.2 - Attributes and Behaviors

- A class defines the **attributes** (data) and **behaviors** (methods) that all objects of that class will have.
    - Objects are specific **instances** of the class that have their own values for the attributes,
    - **Attributes** are data or properties that an object knows about itself, such as its color and size.
    - **Behaviors** are the things that objects can do.

- Attributes and behaviors are defined inside of a class, but each object has its own values for the attributes.
    - For example: all cats have different color patterns, but they have that innate attribute of a color. They are all **instances** of a cat with different "values" for their color, or attribute.
        - Examples of other **attributes**: name, width, height, and **methods** such as forward(), backward(), etc.

1.12.3 - Turtle Class

- The turtle class is also a blueprint for turtle objects.
- The **dot operator** (.) runs an object's method. The parentheses after the method name are there in case you need to give the method **arguments** (some data) to do its job.
- A variable like Yertle (turtle name) is a **reference** type similar to how the Turtle holds an object reference.
    - Objects are complex and contain a collection of values called **attributes**, like color and size of the turtle.

1.12.4 - Creating Turtle Objects

- Writing classes like the Turtle class means you can create many objects of that class type.

1.12.5 - Class Hierarchy and Inheritance

- Another concept in object-oriented programming is **inheritance**.
    - Not covered in AP CSA, but still very useful.
- **Inheritance** is a way to create a new class that is based on an existing class.
    - New class → **subclass**, inherits all attributes and behaviors of the existing class, called a **superclass**.
    - In java, all classes are subclasses of the Object superclass.

1.12.6 - Turtle Methods

- No notes to show.

1.12.7 - Coding Challenge : Draw Letters

Java

```
import java.awt.*;

import java.util.*;


public class TurtleLetter

{

    public static void main(String[] args)

    {

        World habitat = new World(300, 300);

        Turtle t = new Turtle(habitat);

        t.setPenWidth(3);

        t.penDown();

        t.forward(100);

        t.turnRight();

        t.forward(50);

        t.turnRight();

        t.forward(50);

        t.turnRight();

        t.forward(50);


        habitat.show(true);

    }
```

```
}
```

1.12.9 - AP Practice

None

```
A student has created a Dog class. The class contains variables to represent the
following.

   A String variable called breed to represent the breed of the dog

   An int variable called age to represent the age of the dog

   A String variable called name to represent the name of the dog
```

- The object pet is declared as type Dog. Which of the following descriptions is accurate?
    - A. An attribute of the name object is String.
    - **B. An attribute of the pet object is name.**
    - C. An instance of the pet class is Dog.
    - D. An attribute of the Dog instance is pet.
    - E. An instance of the Dog object is a pet.

None

```
A student has created a Party class. The class contains variables to represent the
following.

   An int variable called numOfPeople to represent the number of people at the
      party.

   A boolean variable called discoLightsOn to represent whether the disco ball is
      on.

   A boolean variable called partyStarted to represent whether the party has
      started.
```

- The object myParty is declared as type Party. Which of the following descriptions is accurate?
    - A. boolean is an attribute of the myParty object.
    - B. myParty is an attribute of the Party class.
    - **C. myParty is an instance of the Party class.**
    - D. myParty is an attribute of the Party instance.
    - E. numOfPeople is an instance of the Party object.

1.13 - Creating and Initializing Objects: Constructors

- Java class defines what an object is (attributes) and what they do (behaviors). Each class has **constructors** → used to initialize the attributes of a newly created object.
    - Have the same name as a class.
- Created through new keyword + class name → (new ClassName()).
    - Saved as a **reference type** → holds an object reference or null if there is no object.
- Example:
    - World habitat = new World();
    - Turtle t = new Turtle(habitat);

1.13.1 - The World Class Constructors

- More than one constructor defined in a class → **overloading** the constructor.
- An **argument** is a value passed onto a constructor.
    - Used to initialize the attributes of an object.
- A **no-argument** constructor will utilize default parameters inside of its class definition.

1.13.2 - The Turtle Class Constructors

- The Turtle class has multiple constructors → if no arguments are provided, it will start the Turtle object in the middle of the world. (default)
    - It's important to note that the order of arguments matter.

1.13.3 - Object Variables and References

- New objects saved in variables of a **reference type** which holds a reference to an object.
    - A **reference** is a way to find the object in memory → tracking a number that you can use, like tracking the location of a package in the mail.
    - A special reference value **null** (meaning none) can be used when a variable doesn't refer to any object.
    - For example: Turtle t1 = null; is valid syntax for a variable that doesn't refer to any object. It can be used for debugging or as a placeholder value that can change later on.

1.13.4 - Constructor Signatures

- When using a class that has already been written from a **library** that you can import (such as Turtle) → look up how to use the constructors and methods in the documentation for that class.
    - Documentation will list the **signatures** (or headers) of the constructors/methods which will tell you their name + parameter list. The **parameter list**, in the **header** of a constructor is an ordered list of variable declarations which include data types.
- Constructors are **overloaded** when there are multiple constructors, but have different signatures.
    - Can differ in number, type, and/or order of parameters.

1.13.5 - Arguments, Parameters, and Call by Value

- When a constructor like Date(2005, 9, 1) is called, the **parameters** (variables used inside a class) are set to the copies of the **arguments** (actual values in the constructor).
    - This **call by value** means copies of argument values are passed onto the constructor. (Different from pass by reference)
    - Flow of execution shifts towards anything inside the class after the constructor is initialized.

1.13.6 - Coding Challenge: Custom Turtles

Java

```
import java.awt.*;

import java.util.*;



public class CustomTurtleRunner

{

    public static void main(String[] args)

    {

        World world1 = new World(400, 400);



        // 1. Change the constructor call below to create a large

        // 150x200 CustomTurtle with a green body (Color.green)

        // and a blue shell (Color.blue) at position (150,300).

        // Move it forward to see it.

        CustomTurtle turtle1 = new CustomTurtle(150, 300, world1, Color.green,
Color.blue, 150, 200);

        turtle1.forward();



        // 2. Create a small 25x50 CustomTurtle with a red body

        // and a yellow shell at position (350,200)

        // Move it forward to see it.

        CustomTurtle turtle2 = new CustomTurtle(350, 200, world1, Color.red,
Color.yellow, 25, 50);
```

```
        turtle2.forward();
        // 3. Create a CustomTurtle of your own design
        CustomTurtle turtle3 = new CustomTurtle(100, 100, world1, Color.orange,
Color.black, 100, 300);
        turtle3.forward();
        world1.show(true);
    }
}

class CustomTurtle extends Turtle
{
    private int x;
    private int y;
    private World w;
    private Color bodycolor;
    private Color shellcolor;
    private int width;
    private int height;

    /**
     * Constructor that takes the model display
     *
     * @param modelDisplay the thing that displays the model or world
```

```
     */
    public CustomTurtle(ModelDisplay modelDisplay)
    {
        // let the parent constructor handle it
        super(modelDisplay);
    }


    /**
     * Constructor that takes the model display to draw it on and custom
     * colors and size
     *
     * @param m the world
     * @param body : the body color
     * @param shell : the shell color
     * @param w: width
     * @param h: height
     */
    public CustomTurtle(
            ModelDisplay m, Color body, Color shell, int w, int h)
            {
        // let the parent constructor handle it
        super(m);
        bodycolor = body;
```

```
        setBodyColor(body);
        shellcolor = shell;
        setShellColor(shell);
        height = h;
        width = w;
        setHeight(h);
        setWidth(w);
    }

    /**
     * Constructor that takes the x and y and a model display to draw it on
     * and custom colors and size
     *
     * @param x the starting x position
     * @param y the starting y position
     * @param m the world
     * @param body : the body color
     * @param shell : the shell color
     * @param w: width
     * @param h: height
     */
    public CustomTurtle(
            int x,
```

```
            int y,
            ModelDisplay m,
            Color body,
            Color shell,
            int w,
            int h)
            {
        // let the parent constructor handle it
        super(x, y, m);
        bodycolor = body;
        setBodyColor(body);
        shellcolor = shell;
        setShellColor(shell);
        height = h;
        width = w;
        setHeight(h);
        setWidth(w);
    }
}
```

1.13.7 - Summary

- (AP 1.13.A.1) A class contains **constructors** that are called with the keyword new to create objects and initialize its attributes.

- (AP 1.13.A.1) **Constructors** have the same name as the class.
- (AP 1.13.C.1) An object is typically created using the keyword new followed by a call to one of the class's constructors. new ClassName() creates a new object of the specified class and calls a constructor.
- (AP 1.13.B.1) The new object is saved in a variable of a **reference type** which holds an object reference or null if there is no object.
- (AP 1.13.A.2) A **constructor signature** consists of the constructor's name, which is the same as the class name, and the ordered list of parameter types.
- (AP 1.13.A.2) The **parameter list**, in the header of a constructor, lists the types of the values that are passed and their variable names.
- (AP 1.13.A.3) Constructors are said to be **overloaded** when there are multiple constructors with different signatures. They must differ in the number, type, or order of parameters.
- A **no-argument constructor** is a constructor that doesn't take any passed in values (arguments).
- (AP 1.13.C.2) **Parameters** allow constructors to accept values to establish the initial values of the attributes of the object.
- (AP 1.13.C.3) A constructor **argument** is a value that is passed into a constructor when the constructor is called. The arguments passed to a constructor must be compatible in order and number with the types identified in the parameter list in the constructor signature.
- (AP 1.13.C.3) When calling constructors, arguments are passed using call by value. **Call by value** initializes the parameters with copies of the arguments.
- (AP 1.13.C.4) A constructor call interrupts the sequential execution of statements, causing the program to first execute the statements in the constructor before continuing. Once the last statement in the constructor has been executed, the flow

of control is returned to the point immediately following where the constructor was called.

1.13.8 - AP Practice

Java

```
public class Cat

{

    private String color;

    private String breed;

    private boolean isHungry;


    public Cat()

    {

        color = "unknown";

        breed = "unknown";

        isHungry = false;

    }


    public Cat(String c, String b, boolean h)

    {

        color = c;

        breed = b;

        isHungry = h;

    }

}


I.   Cat a = new Cat();
```

```
II.  Cat b = new Cat("Shorthair", true);

III. String color = "orange";

     boolean hungry = false;

     Cat c = new Cat(color, "Tabby", hungry);
```

- Consider the following class. Which of the following successfully creates a new Cat object?
    - A. I only
    - B. I and II
    - **C. I and III**
    - D. I, II, and III
    - E. II and III

Java

```
public class Movie

{

    private String title;

    private String director;

    private double rating;

    private boolean inTheaters;


    public Movie(String t, String d, double r)

    {

        title = t;

        director = d;

        rating = r;

        inTheaters = false;
```

```
    }


    public Movie(String t)
    {
        title = t;
        director = "unknown";
        rating = 0.0;
        inTheaters = false;
    }
}
```

- Consider the following class. Which of the following code segments will construct a Movie object m with a title of “Lion King” and rating of 8.0?
    - A. Movie m = new Movie(8.0, "Lion King");
    - B. Movie m = Movie("Lion King", 8.0);
    - C. Movie m = new Movie();
    - **D. Movie m = new Movie("Lion King", "Disney", 8.0);**
    - E. Movie m = new Movie("Lion King");

1.14 - Calling Instance Methods

- **Methods** define the behavior and actions that an object can do in object-oriented programming.
    - These methods can be called **instance methods** or **object methods** because they are called using an instance or object of the class.

1.14.1 - Class methods vs. Instance Methods

- We learned how to call **class methods** (or **static methods**) before using the dot operator, but we’ll learn more about **instance methods** that are called using an object of the class. (not static methods)

1.14.2 - Method Signatures

- The **method signature** defines the method's name and the number and types of parameters it takes.
    - Usually defined after the instance variable (attributes) and constructors in a class.

1.14.3 - Method Calls

- To use an object's method, use the object name and the dot (.) operator followed by the method name. Object methods work with the **attributes** of the object, such as the direction the turtle is heading or its position.
    - If creating and object → object.method();
    - If using it inside a class → method.();
- You must ensure that an object has been created and initialized. If you declare an object reference without initializing it, the value will be null. This means that it doesn't reference an object → if you call a method with value null, you will get a **NullPointerException** error, where a **pointer** is another name for a reference.

1.14.4 - Methods Calls with Arguments

- We've used simple **methods** such as forward or turnRight, but these methods can also have specific **arguments** or **parameters** such as the exact amount of pixels to move forward or turn right.
- When creating a method, the variables you define are its **parameters**. When calling the method to do its job, you give or pass in **arguments** to it that are then saved in the parameter variables.
- Methods are **overloaded** when there are multiple methods with the same name but a different **method signature**, or a different number or type of parameters.
    - Useful as forward() and forward(100) accomplish the same thing and don't throw off errors. You can't have default parameters in Java.

1.14.5 - Methods that Return Values

- All methods thus far are **void methods** with void return type → don't return anything.
- By contrast, methods with a return type other than void are called **non-void** methods.
    - They **return** a value that the code calling the method can use.
    - Non-void methods typically don't have effects, they just compute and return a value. Void methods usually do things, while non-void methods produce values.
- A simple kind of method that returns a value is formally called an **accessor** because it accesses a value in an object → in the real world it's called **getters**.
    - A method that takes no arguments and has a non-void return type. Almost always started with a name of get, such as getWidth, getHeight, etc.
- Further in-detail explanation later on in Unit 3, but should be able to trace through method calls.
- Notice that **return statements** return a value that matches the declared return type of the method.

1.14.6 - Coding Challenge : Turtle House

```
Java

import java.awt.*;

import java.util.*;



public class TurtleHouse

{

    public static void main(String[] args)

    {

        // I got lazy
```

```
World world = new World(300, 300);
Turtle t = new Turtle(world);

t.setPenColor(Color.ORANGE);
t.penUp();
t.moveTo(100, 200);
t.penDown();
for(int i=0; i<4; i++) {
    t.forward(100);
    t.turnRight();
}

t.setPenColor(Color.RED);
t.penUp();
t.moveTo(100, 200);
t.penDown();
t.moveTo(150, 100);
t.moveTo(200, 200);
t.moveTo(100, 200);

t.setPenColor(Color.BLUE);
t.penUp();
t.moveTo(140, 200);
```

```
        t.penDown();
        for(int i=0; i<2; i++) {
            t.forward(20);
            t.turnRight();
            t.forward(40);
            t.turnRight();
        }

        t.setPenColor(Color.CYAN);
        t.penUp();
        t.moveTo(110, 160);
        t.penDown();
        for(int i=0; i<4; i++) {
            t.forward(20);
            t.turnRight();
        }

        t.penUp();
        t.moveTo(170, 160);
        t.penDown();
        for(int i=0; i<4; i++) {
            t.forward(20);
            t.turnRight();
```

```
        }
        t.turn(90);
        t.forward(100);
        world.show(true);
    }
}
```

1.14.7 - Summary

- **Instance methods** define the behavior and actions that an object can perform.
- (AP 1.14.A.1) **Instance methods** are called on objects of the class.
- (AP 1.14.A.1) The dot operator is used along with the object name to **call** instance methods, for example **object.method();**
- (AP 1.14.A.2) A method call on a null reference will result in a NullPointerException.
- Some methods take **arguments** that are placed inside the parentheses **object.method(arguments)**.
- A **method signature** is the method name followed by the parameter list which gives the type and name for each parameter. Note that methods do not have to take any parameters, but you still need the parentheses after the method name.
- The method call arguments must match the method signature in number, order, and type.
- A **method** call interrupts the sequential execution of statements, causing the program to first execute the statements in the method or constructor before continuing. Once the last statement in the method or constructor has been executed or a return statement is executed, the flow of control is returned to the point immediately following the method or constructor call.
- **Non-void methods** are methods that return values. You should do something with the return value, such as assigning it to a variable, using it in an expression, or printing it.

1.14.8 - AP Practice

Java

```java
public class Party

{

    private int numInvited;

    private boolean partyCancelled;



    public Party()

    {

        numInvited = 1;

        partyCancelled = false;

    }



    public void inviteFriend()

    {

        numInvited++;

    }



    public void cancelParty()

    {

        partyCancelled = true;

    }

}
```

- Assume that a Party object called myParty has been properly declared and initialized in a class other than Party. Which of the following statements are valid?
    - **A. myParty.cancelParty();**
    - B. myParty.inviteFriend(2);
    - C. myParty.endParty();
    - D. myParty.numInvited();
    - E. System.out.println(myParty.cancelParty());

```java
public class Cat
{
    public void meow()
    {
        System.out.print("Meow ");
    }

    public void purr()
    {
        System.out.print("purr");
    }

    public void welcomeHome()
    {
        purr();
        meow();
    }
```

```
    /* Constructors not shown */
}
```

- Which of the following code segments, if located in a method in a class other than Cat, will cause the message “Meow purr” to be printed?
    - A.
        - Cat a = new Cat();
        - Cat.meow();
        - Cat.purr();
    - B.
        - Cat a = new Cat();
        - a.welcomeHome();
        - content_copy
    - **C.**
        - **Cat a = new Cat();**
        - **a.meow();**
        - **a.purr();**
    - D.
        - Cat a = new Cat().welcomeHome();
    - E.
        - Cat a = new Cat();
        - a.meow();

```
Java
public class Liquid
{
    private double boilingPoint;
    private double freezingPoint;
```

```
    private double currentTemp;


    public Liquid()
    {
        currentTemp = 50;
    }


    public void lowerTemp()
    {
        currentTemp -= 10;
    }


    public double getTemp()
    {
        return currentTemp;
    }
}
// Assume that the following code segment appears in a class other than Liquid.
Liquid water = new Liquid();
water.lowerTemp();
System.out.println(water.getTemp());
```

- What is printed as a result of executing the code segment? (If you get stuck, try this visualization to see this code in action.)
    - A. -10

- B. 50
- C. water.getTemp()
- D. The code will not compile.
- **E. 40.0**

```java
public void splitPizza(int numOfPeople)
{
    int slicesPerPerson = 8/numOfPeople;
    /* INSERT CODE HERE */
}

public void printSlices(int slices)
{
    System.out.println("Each person gets " + slices + " slices each");
}
```

- Which of the following lines would go into /* INSERT CODE HERE */ in the method splitPizza in order to call the printSlices method to print the number of slices per person correctly?
  - **A. printSlices(slicesPerPerson);**
  - B. printSlices(numOfPeople);
  - C. printSlices(8);
  - D. splitPizza(8);
  - E. splitPizza(slicesPerPerson);

1.15 - Strings

- **Strings** in Java are objects of the String class → sequence of characters that store names, addresses, or messages.

1.15.1 - String References

- Object variables **refer** to objects in memory → way to actually find the object.
- Two ways of creating string objects.
    - String greeting = new String(“Hello”);
    - String greeting = “Hello”; or a **string literal**.
- String is the java.lang.String class, where java.lang is the **package** name. Every object in Java knows the class that created it, and likewise every class knows its **parent** class.
    - A class inherits object fields and methods from their parent class. So when it prints java.lang.Object, it is because the parent class (**superclass**) of the String class is the object class. All of them inherit object classes at some point.

1.15.2 - String Operators - Concatenation

- Strings can be added to one another through the + or += operator, also called **concatenation**.
- A string object is **immutable** → once created, it cannot be changed.
- You can add other items like integers or booleans when concatenating them to a String. All objects inherit a toString method from the Object class that returns a String representation of the object and many classes **override** it to produce a useful human-readable value.

1.15.3 - String Index and Length

- A string holds characters in a sequence → each character is a position or **index**. An **index** number is associated with a position in a string and starts at 0.
    - Last character = **length** - 1, accessing indices outside of this range results in an **IndexOutOfBoundsException**.

1.15.4 - String Methods

- You only need to know the following String methods for the exam:

- **int length()** method returns the number of characters in the string, including spaces and special characters like punctuation.
- **String substring(int from, int to)** method returns a new string with the characters in the current string starting with the character at the from index and ending at the character before the to index (if the to index is specified, and if not specified it will contain the rest of the string).
- **int indexOf(String str)** method searches for the string str in the current string and returns the index of the beginning of str in the current string or -1 if it isn't found.
- **int compareTo(String other)** returns a negative value if the current string is less than the other string alphabetically, 0 if they have the same characters in the same order, and a positive value if the current string is greater than the other string alphabetically.
- **boolean equals(Object other)** returns true when the characters in the current string are the same as the ones in the other string. This method is inherited from the Object class, but is **overridden** which means that the String class has its own version of that method.

1.15.5 - String Methods: length, substring, indexOf

- No notes to show.

1.15.6 - CompareTo and Equals

- We can compare primitive types like int and double using operators like ==, <, or >. To compare strings, we use equals and compareTo.
  - Method compareTo compares two strings character by character. If they are equal, it returns 0. If the first string is alphabetically ordered before the second string, it returns a negative number, and if the opposite is true, then it returns a positive number.
- Equals method compares two strings character by character and returns true or false. Both compareTo and equals are case-sensitive. There are case-insensitive versions, compareToIgnoreCase and equalsIgnoreCase.

1.15.7 - Common Mistakes with Strings

- Common mistakes with strings:
    - Thinking that substrings include the character at the last index when they don't.
    - Thinking that strings can change when they can't. They are immutable.
    - Trying to access part of a string that is not between index 0 and length -1. This will throw an IndexOutOfBoundsException.
    - Trying to call a method like indexOf on a string reference that is null. You will get a null pointer exception.
    - Using == to test if two strings are equal. This is actually a test to see if they refer to the same object. Usually you only want to know if they have the same characters in the same order. In that case you should use equals or compareTo instead.
    - Treating upper and lower case characters the same in Java. If s1 = "Hi" and s2 = "hi" then s1.equals(s2) is false.

1.15.8 - Coding Challenge : Pig Latin

```
Java

import java.util.Scanner;


public class PigLatin {


    public static String pigLatin(String word) {
        // Got lazy
        String vowels = "aeiouAEIOU";
        int firstVowelIndex = -1;


        for (int i = 0; i < word.length(); i++) {
```

```
            if (vowels.indexOf(word.charAt(i)) != -1) {
                firstVowelIndex = i;
                break;
            }
        }

        if (firstVowelIndex == -1) {
            // No vowels found, just return the word with "ay"
            return word + "ay";
        } else if (firstVowelIndex == 0) {
            // Word starts with a vowel
            return word + "way";
        } else {
            // Word starts with a consonant
            String startPart = word.substring(firstVowelIndex);
            String endPart = word.substring(0, firstVowelIndex);
            return startPart + endPart + "ay";
        }
    }

    public static void main(String[] args) {
        // Do not change main!
        // Write your code in the pigLatin method above.
```

```
        Scanner scan = new Scanner(System.in);
        String word = scan.nextLine();
        System.out.println(word + " in Pig Latin is " + pigLatin(word));
        scan.close();
    }
}
```

1.15.9 - Summary

- (AP 1.15.A.1) A String object represents a sequence of characters and can be created by using a string literal.
- (AP 1.15.A.2) The String class is part of the java.lang package. Classes in the java.lang package are available by default.
- String objects can be created by using string literals (String s = “hi”;) or by calling the String class constructor (String t = new String(“bye”);).
- (AP 1.15.A.3) A String object is **immutable**, meaning once a String object is created, its attributes cannot be changed. Methods called on a String object do not change the content of the String object.
- (AP 1.15.A.4) Two String objects can be concatenated together or combined using the + or += operator, resulting in a new String object.
- (AP 1.15.A.4) A primitive value can be concatenated with a String object. This causes the implicit conversion of the primitive value to a String object.
- (AP 1.15.A.5) A String object can be concatenated with any object, which implicitly calls the object’s toString method (a behavior which is guaranteed to exist by the inheritance relationship every class has with the Object class). An object’s toString method returns a string value representing the object. Subclasses of Object often **override** the toString method with class-specific implementation. Method overriding occurs when a public method in a subclass has the same method signature as a public method in the superclass, but the behavior of the method is specific to the

subclass. Overriding the toString method of a class is outside the scope of the AP CSA exam.

- **index** - A number that represents the position of a character in a string. The first character in a string is at index 0.
- **length** - The number of characters in a string.
- **substring** - A new string that contains a copy of part of the original string.
- (AP 1.15.B.1) A String object has index values from 0 to one less than the length of the string. Attempting to access indices outside this range will result in an IndexOutOfBoundsException.
- (AP 1.15.B.2) The following String methods and constructors, including what they do and when they are used, are part of the AP CSA Java Quick Reference Sheet that you can use during the exam:
    - **String(String str)** : Constructs a new String object that represents the same sequence of characters as str.
    - **int length()** : returns the number of characters in a String object.
    - **String substring(int from, int to)** : returns the substring beginning at index from and ending at index (to -1).
    - **String substring(int from)** : returns substring(from, length()).
    - **int indexOf(String str)** : searches for str in the current string and returns the index of the first occurrence of str; returns -1 if not found.
    - **boolean equals(Object other)** : returns true if this (the calling object) is equal to other; returns false otherwise. Using the equals method to compare one String object with an object of a type other than String is outside the scope of the AP CSA exam.
    - **int compareTo(String other)** : returns a value < 0 if this is less than other; returns zero if this is equal to other; returns a value > 0 if this is greater than other. Strings are ordered based upon the alphabet.
- str.substring(index, index + 1) returns a single character at index in string str.

1.15.10 - AP Practice

```java
String s1 = new String("hi there");
```

```
int pos = s1.indexOf("e");

String s2 = s1.substring(0,pos);
```

- What is the value of s2 after the following code executes?
    - **A. hi th**
    - B. hi the
    - C. hi ther
    - D. hi there

```
Java

String s1 = "Hi";

String s2 = s1.substring(0,1);

String s3 = s2.toLowerCase();
```

- What is the value of s1 after the following code executes?
    - **A. Hi**
    - B. hi
    - C. H
    - D. h

1.15.11 - String Methods Game

- No notes to show

1.15.12 - Review/Practice for Unit 1 on Using Objects.

1.16 - Unit Summary 1a (1.1-1.6)

1.16.1 - Concept Summary

- **Compiler** - Software that translates the Java source code into the Java class file which can be run on the computer.

- **Compiler or syntax error** - An error that is found during the compilation.
- **Main method** - Where execution starts in a Java program.
- **Variable** - A name associated with a memory location in the computer.
- **Declare a Variable** - Specifying the type and name for a variable. This sets aside memory for a variable of that type and associates the name with that memory location.
- **Initializing a Variable** - The first time you set the value of a variable.
- **data type** - determines the size of memory reserved for a variable, for example int, double, boolean, String.
- **integer** - a whole number like 2 or -3
- **boolean** - An expression that is either true or false.
- **Camel case** - One way to create a variable name by appending several words together and uppercasing the first letter of each word after the first word (myScore).
- **Casting a Variable** - Changing the type of a variable using (type) name.
- **Operator** - Common mathematical symbols such as + for addition and * for multiplication.
- **Compound assignment or shortcut operators** - Operators like x++ which means x = x + 1 or x *=y which means x = x * y.
- **remainder** - The % operator which returns the remainder from one number divided by another.
- **arithmetic expression** - a sequence of operands and operators that describe a calculation to be performed, for example 3*(2 + x)
- **operator precedence** - some operators are done before others, for example *, /, % have precedence over + and -, unless parentheses are used.

1.16.2 - Java Keyword Summary

- **boolean** - used to declare a variable that can only have the value true or false.
- **double** - used to declare a variable of type double (a decimal number like 3.25).
- **int** - used to declare a variable of type integer (a whole number like -3 or 235).
- **String** - used to declare a variable of type String which is a sequence of characters or text.
- **System.out.print()** - used to print output to the user
- **System.out.println()** - used to print output followed by a newline to the user

- **=** - used for assignment to a variable
- **+, -, *, /, %** - arithmetic operators

1.16.3 - Vocabulary Practice

- No notes to show.

1.16.4 - Common Mistakes

- forgetting that Java is case sensitive - myScore is not the same as myscore.
- forgetting to specify the type when declaring a variable (using name = value; instead of type name = value;)
- using a variable name, but never declaring the variable.
- using the wrong name for the variable. For example calling it studentTotal when you declare it, but later calling it total.
- using the wrong type for a variable. Don't forget that using integer types in calculations will give an integer result. So either cast one integer value to double or use a double variable if you want the fractional part (the part after the decimal point).
- using == to compare double values. Remember that double values are often an approximation. You might want to test if the absolute value of the difference between the two values is less than some amount instead.
- assuming that some value like 0 will be smaller than other int values. Remember that int values can be negative as well. If you want to set a value to the smallest possible int values use Integer.MIN_VALUE.

1.17 - Toggle Code Practice 1a (1.1-1.6)

- No notes to show.

1.18 - Coding Practice 1a (1.1-1.6)

- No notes to show.

1.19 - Multiple Choice Exercises for Unit 1a (1.1-1.6)

- No notes to show.

1.20 - Unit Summary 1b (1.7-1.15)

1.20.1 - Concept Summary

- **class** - defines a new data type. It is the formal implementation, or blueprint, of the attributes and behaviors of the objects of that class.
- **object** - a specific instance of a class with defined attributes. Objects are declared as variables of a class type.
- **constructors** - code that is used to create new objects and initialize the object's attributes.
- **new** - keyword used to create objects with a call to one of the class's constructors.
- **instance variables** - define the attributes for objects.
- **methods** - define the behaviors or functions for objects.
- **dot (.) operator** - used to access an object's methods.
- **parameters (arguments)** - the values or data passed to an object's method inside the parentheses in the method call to help the method do its job.
- **return values** - values returned by methods to the calling method.
- **immutable** - String methods do not change the String object. Any method that seems to change a string actually creates a new string.
- **wrapper classes** - classes that create objects from primitive types, for example the Integer class and Double class.

1.20.2 - Java Keyword Summary

- **new** is used to create a new object.
- **null** is used to indicate that an object reference doesn't refer to any object yet.
- The following String methods and constructors, including what they do and when they are used, are part of the Java Quick Reference in the AP exam:
    - **String(String str)** : Constructs a new String object that represents the same sequence of characters as str.
    - **int length()** : returns the number of characters in a String object.
    - **String substring(int from, int to)** : returns the substring beginning at index from and ending at index (to -1). The single element substring at position index can be created by calling substring(index, index + 1).

    - **String substring(int from)** : returns substring(from, length()).
    - **int indexOf(String str)** : returns the index of the first occurrence of str; returns -1 if not found.
    - **boolean equals(Object other)** : returns true if this (the calling object) is equal to other; returns false otherwise.
    - **int compareTo(String other)** : returns a value < 0 if this is less than other; returns zero if this is equal to other; returns a value > 0 if this is greater than other.
- The following Integer methods and constructors, including what they do and when they are used, are part of the Java Quick Reference.
    - Integer(value): Constructs a new Integer object that represents the specified int value.
    - Integer.MIN_VALUE : The minimum value represented by an int or Integer.
    - Integer.MAX_VALUE : The maximum value represented by an int or Integer.
    - int intValue() : Returns the value of this Integer as an int.
- The following Double methods and constructors, including what they do and when they are used, are part of the Java Quick Reference Guide given during the exam:
    - Double(double value) : Constructs a new Double object that represents the specified double value.
    - double doubleValue() : Returns the value of this Double as a double.
- The following static Math methods are part of the Java Quick Reference:
    - **int abs(int)** : Returns the absolute value of an int value (which means no negatives).
    - **double abs(double)** : Returns the absolute value of a double value.
    - **double pow(double, double)** : Returns the value of the first parameter raised to the power of the second parameter.
    - **double sqrt(double)** : Returns the positive square root of a double value.
    - **double random()** : Returns a double value greater than or equal to 0.0 and less than 1.0 (not including 1.0!).
- **(int)(Math.random()*range) + min** moves the random number into a range starting from a minimum number. The range is the (max number - min number + 1). For example, to get a number in the range of 5 to 10, use the range 10-5+1 = 6 and the min number 5: (int)(Math.random()*6) + 5).

1.20.3 - Vocabulary Practice

- No notes to show.

1.20.4 - Common Mistakes

- Forgetting to declare an object to call a method from main or from outside of the class, for example object.method();
- Forgetting () after method names when calling methods, for example object.method();
- Forgetting to give the right parameters in the right order to a method that requires them.
- Forgetting to save, print, or use the return value from a method that returns a value, for example int result = Math.pow(2,3);
- Using == to test if two strings or objects are equal. This is actually a test to see if they refer to the same object. Usually you only want to know if they have the same characters in the same order. In that case you should use equals(String) or compareTo(String) instead.
- Treating upper and lower case characters the same in Java. If s1 = "Hi" and s2 = "hi" then s1.equals(s2) is false.
- Thinking that substrings include the character at the last index when they don't.
- Thinking that strings can change when they can't. They are immutable.
- Trying to call a method like str1.indexOf(str2) with a string reference str1 that is null. You will get a null pointer exception.

1.21 - Toggle Code Practice

- No notes to show.

1.22 - Coding Practice 1b (1.7-1.15)

- No notes to show.

1.23 - Multiple Choice Exercises for Unit 1b (1.9-1.15)

- No notes to show.

1.24 - Practice Test for Objects (1.12-1.14)

- No notes to show.

1.25 - Java Swing GUIs (optional)

- No notes to show

1.26 - Unit 1 Free Response Question (FRQ) Practice

- No notes to show

Unit 2 - Selection and Iteration

2.1 - Algorithms with Selection and Repetition

- All algorithms have sequences of steps.
    - So far we've had only sequential algorithms, but could be more complex.
- **Sequencing**, **selection**, and **repetition**. Algorithms can have selection through decision making, and repetition, via looping.
    - It's proven that all algorithms are fundamentally one of these three control structures.

2.1.1 - Selection

- Process of making a choice based on a true/false decision. This (selection, or branching) occurs when a choice of how the execution of an algorithm will proceed based on a boolean value. In other words, it may be different based on a condition.

2.1.2 - Repetition

- Process that is repeated until a desired outcome is reached.
    - Achieved through loops. A **loop** allows an algorithm to repeat actions until a condition is met. Also called **iteration**.

2.1.3 - Algorithms with Pseudocode and Flowcharts

- It's important to plan your solution before writing code.
- **Pseudocode** is a simplified and informal way of describing steps in an algorithm in a human language like English but follows programming concepts.

- **Flowcharts** are diagrams that represent the steps. Selection usually is represented as a triangle in a flowchart + arrows are used to show repetition.

2.1.4 - Coding Challenge : Algorithms

2.1.5 - Summary

- (AP 2.1.A.1) The building blocks of algorithms include sequencing, selection, and repetition.
- (AP 2.1.A.2) Algorithms can contain selection, through decision making, and repetition, via looping.
- (AP 2.1.A.3) Selection occurs when a choice of how the execution of an algorithm will proceed is based on a true or false decision.
- (AP 2.1.A.4) Repetition is when a process repeats itself until a desired outcome is reached.
- (AP 2.1.A.5) The order in which sequencing, selection, and repetition are used contributes to the outcome of the algorithm.

2.2 - Boolean Expressions

2.2.1 - Testing Equality (==)

- Relational operators == and != → used to compare values, return true/false boolean values.
- Two ways to declare object instances:
  - **References** - Creating an object instance from a class.
  - **Aliases** - Set an object value to a previously created object.

2.2.2 - Relational Operators (<, >)

- **Relational operators** below are used to compare numeric values or arithmetic expressions.
- Some programming languages use < to compare strings → Java uses them for numbers only.
- compareTo() and equals() methods are in Java.

- **Boolean** variables have **true**/**false** values →used to compare two variables or compare a variable against a constant value/expression.
    - Can check if a number is positive/negative.

2.2.3 - Testing with remainder (%)

- Useful in coding → returns the remainder of a division between two numbers.
- Not a modulo operator → returns the remainder.

2.2.4 - Coding Challenge : Prime Numbers POGIL

- Use **POGIL** to do the following activities.
- Prime numbers are often used for encryption as guessing the factors for a large number is often tedious to impossible.

2.2.5 - Summary

- (AP 2.2.A.1) Values or expressions can be compared using the relational operators == and != to determine whether the values are the same. With primitive types, this compares the actual primitive values. With reference types, this compares the object references.
- (AP 2.2.A.2) Numeric values or expressions can be compared using the relational operators (<, >, <=, >=) to determine the relationship between the values.
- (AP 2.2.A.3) An expression involving relational operators evaluates to a Boolean value of true or false.
- The remainder operator % can be used to test for divisibility by a number. For example, num % 2 == 0 can be used to test if a number is even.

2.2.6 - AP Practice

- No notes to show.

2.2.7 - Relational Operators Practice Game

- **Relationals** can be tricky, make sure to evaluate each answer choice thoroughly.

2.3 - If Statements

- **If statements** are found in all programming languages → way to branch and have different paths in an algorithm.
    - If a statement is a type of **selection** → changes sequential execution.
    - Affects flow of control → executing different code segments → based on **Boolean expression**.

2.3.1 - One-way selection

- Used when there is a segment of code (in the **body** of execution)
    - Executed only when it is true.
    - If false → body of statement is skipped + program skips execution.
- Open and curly braces used to group blocks of statements → body.
    - Always put them all under one statement.
    - AP Exam you will use curly braces.

2.3.2 - Relational Operators in If Statements

- Most if statements have a boolean condition that uses relational operators like ==, !=, <, >, <=, >= → we saw in the last lesson.

2.3.3 - Two-way selection

- You can use the **if** keyword followed by a statement/block of statements → use **else** keyword followed by another statement.
- Two-way selection is used when there are two segments of code merged into one.
    - Only executed when the Boolean expression is true, and another segment when the Boolean expression is false.

2.3.4 - Common Errors with If Statements

- Common rules to follow to avoid common errors:
    - Always use curly braces ({ and }) to enclose the block of statements under the if condition. Java doesn't care if you indent the code—it goes by the { }.
    - Don't put in a semicolon ; after the first line of the if statement, if (test);. The if statement is a multiline block of code that starts with the if condition and then { the body of the if statement }.

- Always use ==, not =, in the condition of an if statement to test a variable. One = assigns, two == tests!
- The else statement matches with the closest if statement. If you want to match an else with a different if statement, you need to use curly braces to group the if and else together.

2.3.5 - Coding Challenge: Magic 8 Ball

```java
public class Magic8Ball

{

    public static void printRandomResponse()

    {

        // 1. Get a random number from 1 to 8

        int num = (int)(Math.random() * 8) + 1;


        // 2. Use if statements to test the random number

        if (num == 1)

            System.out.println("Signs point to yes!");

        if (num == 2)

            System.out.println("Outlook not so good.");

        if (num == 3)

            System.out.println("Yes, definitely!");

        if (num == 4)

            System.out.println("Very doubtful.");

        if (num == 5)
```

```
            System.out.println("Ask again later.");
        if (num == 6)
            System.out.println("Without a doubt!");
        if (num == 7)
            System.out.println("Better not tell you now.");
        if (num == 8)
            System.out.println("My sources say no.");
    }

    public static void lucky()
    {
        // 3. Use Math.random() to toss a coin (1 or 2)
        int coin = (int)(Math.random() * 2) + 1;

        // 4. Use if/else to print the result
        if (coin == 1)
            System.out.println("Lucky!");
        else
            System.out.println("No Luck!");
    }

    public static void main(String[] args)
    {
```

```
        String question = "Will it rain tomorrow?";
        System.out.println(question);
        printRandomResponse();
        lucky();
    }
}
```

2.3.6 - Summary

- (AP 2.3.A.1) Selection statements change the sequential execution of statements.
- (AP 2.3.A.2) An **if statement** is a type of selection statement that affects the flow of control by executing different segments of code based on the value of a Boolean expression.
- (AP 2.3.A.3) A one-way selection (if statement) is used when there is a segment of code to execute under a certain condition. In this case, the body is executed only when the Boolean expression is true.
- **if statements** test a boolean expression and if it is true, go on to execute the body which is the following statement or block of statements surrounded by curly braces ({}) like below.

Java

```
// A single if statement
if (boolean expression)
    Do statement;
// A block if statement
if (boolean expression)
```

```
{
   Do Statement1;
   Do Statement2;
   ...
   Do StatementN;
}
```

- Relational operators (==, !=, <, >, <=, >=) are used in boolean expressions to compare values and arithmetic expressions.
- If statements can be followed by an associated **else** part to form a 2-way branch:

```
Java
if (boolean expression)
{
    Do statement;
}
else
{
    Do other statement;
}
```

- (AP 2.3.A.4) A two-way selection (if-else statement) is used when there are two segments of code—one to be executed when the Boolean expression is true and

another segment for when the Boolean expression is false. In this case, the body of the if is executed when the Boolean expression is true, and the body of the else is executed when the Boolean expression is false.

2.3.7 - AP Practice

- No notes to show.

2.4 - Nested if Statements

- If statements can be nested within other if statements. They consist of if, if-else, or if-else-if statements within previous if statements.

2.4.1 - Multiway selection (else if)

- Single if-else statement → 2 branches of code.
- Nested if-else statements → 3 or more branches of code.
    - Used when there are a series of expressions with different segments of code for each condition.
- If no expression evaluates to true + trailing else statement → body of the else is executed.
- Add the **else if** for each possibility after the first **if**, and **else** before the last possibility like below.

2.4.2 - Dangling Else Statements

- Sometimes with nested ifs we find a **dangling else** → potentially belonging to either if statement.
    - Else clauses will always be a part of the closest unmatched if statement, regardless of indentation.
- You can use curly brackets to enclose a nested if.

2.4.3 - Coding Challenge : Adventure

Java

```
import java.util.Scanner;


public class Adventure
{
    private static Scanner scan = new Scanner(System.in);


    public static void main(String[] args)
    {
        // TODO #1. Creative intro text
        System.out.println("You are on a mysterious island surrounded by endless
blue water.");
        System.out.println("There is a path to the forest to the north, "
                           + "the sea to the south, a mountain to the east, "
                           + "and an abandoned village to the west.");
        System.out.println("Which way do you want to go (n,e,s,w)?");


        String command = scan.next(); // use nextLine() in your IDE
        if (command.equals("n"))
        {
            System.out.println("You go north.");
            forest();
        }
        else if (command.equals("s"))
```

```
        {
            System.out.println("You go south.");
            sea();
        }
        else if (command.equals("e"))
        {
            System.out.println("You go east.");
            mountain();
        }
        else if (command.equals("w"))
        {
            System.out.println("You go west.");
            village();
        }
        else
        {
            System.out.println("You can't go in that direction!");
        }

        System.out.println("End of adventure!");
        scan.close();
    }
```

```
  // TODO #3: Complete this method
  public static void forest()
  {
      System.out.println("You enter a dark forest filled with towering trees.");
      System.out.println("You hear rustling in the bushes... maybe an animal?");
      System.out.println("Do you want to walk e (to the sea) or w (to the
village)?");

      String command = scan.next();
      if (command.equals("e"))
      {
          System.out.println("You move east and reach the sea.");
          sea();
      }
      else if (command.equals("w"))
      {
          System.out.println("You move west and find yourself in the ruins of a
village.");
          village();
      }
      else
      {
          System.out.println("The trees block your way. You can’t go that
direction!");
```

```
      }
  }

  // TODO #4: South from main or east from forest goes to the sea
  public static void sea()
  {
      System.out.println("The waves crash against the shore.");
      System.out.println("A small boat is tied to a post. Seagulls circle
overhead.");
      System.out.println("Do you want to go n (back to the island), or w (to the
village)?");

      String command = scan.next();
      if (command.equals("n"))
      {
          System.out.println("You walk back to the center of the island.");
          main(new String[0]);
      }
      else if (command.equals("w"))
      {
          System.out.println("You head west along the shore and find the abandoned
village.");
          village();
      }
```

```
      else
      {
          System.out.println("The sea is too dangerous to cross that way!");
      }
  }

  // TODO #5: Mountain location
  public static void mountain()
  {
      System.out.println("You climb a steep mountain. The air is thin, and you see
the whole island.");
      System.out.println("To the west is the forest. To the south, the sea
sparkles.");
      System.out.println("Where do you want to go (w or s)?");

      String command = scan.next();
      if (command.equals("w"))
      {
          System.out.println("You descend into the forest.");
          forest();
      }
      else if (command.equals("s"))
      {
          System.out.println("You climb down towards the sea shore.");
```

```
            sea();
        }
        else
        {
            System.out.println("The cliff face is too dangerous to continue!");
        }
    }

    // TODO #5: Village location
    public static void village()
    {
        System.out.println("You arrive at a crumbling village. The houses are empty,
"
                           + "but you sense that someone once lived here.");
        System.out.println("From here, you can go e (to the forest) or s (to the
sea).");

        String command = scan.next();
        if (command.equals("e"))
        {
            System.out.println("You travel east into the forest.");
            forest();
        }
        else if (command.equals("s"))
```

```
        {
            System.out.println("You follow a dusty path south to the shore.");
            sea();
        }
        else
        {
            System.out.println("The ruins are impassable in that direction!");
        }
    }
}
```

2.4.4 - Summary

- (AP 2.4.A.1) Nested if statements consist of if, if-else, or if-else-if statements within if, if-else, or if-else-if statements.
- (AP 2.4.A.2) The Boolean expression of the inner nested if statement is evaluated only if the Boolean expression of the outer if statement evaluates to true.
- (AP 2.4.A.3) A multi-way selection (if-else-if) is used when there are a series of expressions with different segments of code for each condition. Multi-way selection is performed such that no more than one segment of code is executed based on the first expression that evaluates to true. If no expression evaluates to true and there is a trailing else statement, then the body of the else is executed.

```
Java

// 3 way choice with else if

if (boolean expression)
```

```
{
   statement1;
}
else if (boolean expression)
{
   statement2;
}
else
{
    statement3;
}
```

2.4.5 - AP Practice

- No notes to show.

2.5 - Compound Boolean Expressions

2.5.1 - And (&&), Or (||), and Not (!)

- Use && as a logical **and** → join two Boolean expressions and the body of the condition will only execute if both are true.
- What if we only want two things to be true? We use the || as the logical **or** to join two Boolean expressions → only one needs to be true.
- The **not** (!) operator can be used to negate a boolean value. It returns the opposite of the expression. If true → false, or if false → true.
    - The order of operations for these 3 values is ! → && → ||.

2.5.2 - Truth Tables

- A **truth table** displays the values involving boolean expressions and the corresponding logical operators.

2.5.3 - Short Circuit Evaluation

- Both && and || use **short circuit evaluation**. This means that the second expression isn't necessarily checked if the result from the first expression is enough to tell if the compound boolean expression is true or false.
    - If the first expression is true on an **or** operator → the entire statement is true.
    - If the first expression is false on an **and** operator → the entire statement is false.

2.5.4 - Coding Challenge : Truth Tables POGIL

```
Java

public class TruthTable

{

    public static void main(String[] args)

    {

        // Test multiple values for these variables

        boolean sunny = false;

        int temperature = 90;

        boolean raining = false;



        // Write an if statement for: If it's sunny,

        //  OR if the temperature is greater than 80

        //     and it's not raining, "Go to the beach!"
```

```
        if ((sunny || temperature > 80) && !(raining)) {
                System.out.println("Go to the beach!");
        }
    }
}
```

2.5.5 - Summary

- (AP 2.5.A.1) Logical operators ! (not), && (and), and || (or) are used with Boolean values.
    - A && B is true if both A and B are true.
    - A || B is true if either A or B (or both) are true.
    - !A is true if A is false.
- (AP 2.5.A.1) ! has precedence (is executed before) && which has precedence over ||. (Parentheses can be used to force the order of execution in a different way.)
- (AP 2.5.A.1) An expression involving logical operators evaluates to a Boolean value.
- (AP 2.5.A.2) **Short-circuit evaluation** occurs when the result of a logical operation using && or || can be determined by evaluating only the first Boolean expression. In this case, the second Boolean expression is not evaluated. (If the first expression is true in an || operation, the second expression is not evaluated since the result is true. If the first expression is false in an && operation, the second expression is not evaluated since the result is false.)

2.5.6 - AP Practice

- No notes to show.

2.5.7 - Boolean Game

- No notes to show.

2.6 - Comparing Boolean Expressions (De Morgan's Laws)

2.6.1 - De Morgan's Laws

- Developed by Augustus De MOrgan in the 1800s.
- Show how to simplify negation of a complex boolean expression → when there are multiple expressions joined by **and** or **or**.
- When you negate one of these expressions, you can simplify it by flipping the operators and end up with an equivalent expression. De Morgan's Laws state the following equivalencies:
    - **Move the NOT inside, AND becomes OR**
    - **Move the NOT inside, OR becomes AND.**
- In relational operators, **to move the NOT, flip the sign**.
    - == → !=
    - < → >=
    - > → <=
    - <= → >
    - >= → <

2.6.2 - Truth Tables

- Though you don't have to memorize De Morgan's Laws, you have to show that two boolean expressions are equivalent.

2.6.3 - Simplifying Boolean Expressions

- You can simplify boolean expressions to create equivalent expressions. For instance:
    - !(x < 3 && y > 2) → !(x < 3) || !(y > 2) → (x >= 3 || y <= 2)

2.6.4 - Comparing Objects

- Comparing objects is different than comparing primitive typed values like numbers.
- Can be very complex and have attribute values or instance variables inside them.
- When comparing two turtle objects, we need a specifically written **equals** method to compare all of these values → strings.

2.6.5 - String Equality

- The **equals** method for string compares two strings letter by letter.
    - Returns true when the two variables refer to the same object.
    - These variables are called **object references** and aliases for the same object.

2.6.6 - Equality with New Strings

- If you use the new keyword to create a string → always creates a new string object. Even if we create two strings containing the same characters, it won't refer to the same object.

2.6.7 - Comparing with null

- One common place to use == or != is when comparing them to **null** to see if they really exist.
    - Short-circuit evaluation is used to avoid errors in case an object doesn't exist.
    - **Short-circuit evaluation** used with && in Java → first part of the if condition is false, it doesn't even have to check the second condition.

2.6.8 - Coding Challenge : Truth and Tracing Tables POGIL

```java
public class EquivalentExpressions

{

    public static void main(String[] args)

    {

        int x = -1; // try with x = -1, x = 0, and x = 1

        System.out.println(!(x == 0 || x >= 1));

        System.out.println(!(x == 0) && !(x >= 1));

        System.out.println((x != 0) && (x < 1));
```

```
    }
}
```

2.6.9 - Summary

- (AP 2.6.A.1) Two Boolean expressions are equivalent if they evaluate to the same value in all cases. Truth tables can be used to prove Boolean expressions are equivalent.
- (AP 2.6.A.2) De Morgan's Laws can be applied to Boolean expressions to create equivalent ones:
    - !(a && b) is equivalent to !a || !b
    - !(a || b) is equivalent to !a && !b
- A negated expression with a relational operator can be simplified by flipping the relational operator to its opposite sign.
    - !(c == d) is equivalent to c != d
    - !(c != d) is equivalent to c == d
    - !(c < d) is equivalent to c >= d
    - !(c > d) is equivalent to c <= d
    - !(c <= d) is equivalent to c > d
    - !(c >= d) is equivalent to c < d
- (AP 2.6.B.1) Two different variables can hold references to the same object. Object references can be compared using == and !=. (Two object references are considered **aliases** when they both reference the same object.)
- (AP 2.6.B.2) An object reference can be compared with null, using == or !=, to determine if the reference actually references an object.
- (AP 2.6.B.3) Classes often define their own equals method, which can be used to specify the criteria for equivalency for two objects of the class. The equivalency of two objects is most often determined using attributes from the two objects.

2.6.10 - AP Practice

- No notes to show.

2.7 - While Loops

- You can set songs to loop → means that it reaches the end and starts over at the beginning.
- A **loop** in programming (**iteration** or **repetition**) is a way to repeat one or more statements.
- The **control structures** in programming allows you to construct algorithms to solve almost any programming problem.
- A while loop executes the body of the loop as long as a boolean condition is true.
    - When it is false → exists the loop and continues with the statements after the body of the while loop.

2.7.1 - Three Steps to Writing a Loop

- The simplest loops are **counter-controlled loops**, where the **loop control variable** is a counter that controls how many times to repeat the loop.
    - 1 - Initialize the loop variable (before the while loop)
    - 2 - Test the loop variable (in the loop header)
    - 3 - Update the loop variable (in the while loop body at the end)

2.7.2 - Tracing Loops

- A good skip to develop is to trace the values of variables and how they change during each iteration of a loop.
- Create a tracing table that keeps track of the variable values each time through the loop.

2.7.3 - Common Errors with Loops

- A common error is to accidentally create an **infinite loop** → never stops because the Boolean condition is always true.
- Another common error is an **off-by-one error** where the loop runs one too many times or one too few times.

2.7.4 - Input-Controlled Loops

- **Input-controlled loop** is used for users to indicate when to stop.
    - The stopping value is called the **sentinel value** for the loop.

2.7.5 - Coding Challenge : Turtle Squares

Java

```
import java.awt.*;

import java.util.*;


public class TurtleDrawSquare

{

    public static void main(String[] args)

    {

        World world = new World(300, 300);

        Turtle yertle = new Turtle(world);


        // Change the following code to use a while loop to draw the square

        // Remember to initialize a counter variable, test it, and increment it.


        int i = 0;

        while (i < 4) {

            yertle.forward();

            yertle.turn(90);

           i++;

        }
```

```
        world.show(true);
    }
}
```

2.7.6 - Summary

- (AP 2.7.A.1) Iteration statements (loops) change the flow of control by repeating a segment of code zero or more times as long as the Boolean expression controlling the loop evaluates to true. Iteration is a form of repetition.
- (AP 2.7.B.1) A **while loop** is a type of iterative statement. In while loops, the Boolean expression is evaluated before each iteration of the loop body, including the first. When the expression evaluates to true, the loop body is executed. This continues until the Boolean expression evaluates to false, whereupon the iteration terminates. Here is the general form of a while loop:

Java

```
// The statements in a while loop run zero or more times,
// determined by how many times the condition is true
int count = 0; // initialize the loop variable
while (count < 10)  // test the loop variable
{
    // repeat this code
    // update the loop variable
    count++;
}
```

- Loops often have a **loop control variable** that is used in the boolean condition of the loop. Remember the 3 steps of writing a loop:
    - Initialize the loop variable
    - Test the loop variable
    - Update the loop variable
- (AP 2.7.A.2) An **infinite loop** occurs when the Boolean expression in an iterative statement always evaluates to true.
- (AP 2.7.A.3) The loop body of an iterative statement will not execute if the Boolean expression initially evaluates to false.
- (AP 2.7.A.4) **Off by one errors** occur when the iteration statement loops one time too many or one time too few.
- **Input-controlled loops** often use a **sentinel value** that is input by the user like "bye" or -1 as the condition for the loop to stop. Input-controlled loops are not on the AP CSA exam, but are very useful to accept data from the user.

2.7.7 - AP Practice

- No notes to show.

2.8 - For Loops

- Another type of iteration is the **for loop**.
    - It is a **count-controlled loop** → does body set amount of times.

2.8.1 - Three Parts of a For Loop

- A for loop combines all 3 parts of writing a loop in one line to initialize, test, and update the loop control variable. The 3 parts are separated by semicolons (;);
    - for (initialize; test; update)
    - The variable being initialized is referred to as a **loop control variable**.
    - Boolean expression is evaluated immediately after the loop control variable is initialized → followed by each execution of the increment statement until it is false.

2.8.2 - Decrementing Loops

- You can also count backwards depending on the iterator.
    - Method **printPopSong** prints the words to a song.

2.8.3 - Coding Challenge : Turtles Drawing Shapes

```java
import java.awt.*;

import java.util.*;


public class TurtleDrawShapes

{


    public static void drawSquare(Turtle t)

    {

        t.setColor(Color.blue);

        // 1. Complete the following for loop to draw a square with 4 sides

        for(int i = 0; i < 4; i++)

        {

            t.forward(100);

            t.turn(90); // 360 / 4 = 90 degrees

        }

    }


    public static void drawTriangle(Turtle t)

    {
```

```
        t.setColor(Color.green);
        // 2. Use a for loop to draw a triangle (3 sides)
        for(int i = 0; i < 3; i++)
        {
            t.forward(100);
            t.turn(120); // 360 / 3 = 120 degrees
        }
    }

    public static void drawPentagon(Turtle t)
    {
        t.setColor(Color.red);
        // 3. Use a for loop to draw a pentagon (5 sides)
        for(int i = 0; i < 5; i++)
        {
            t.forward(100);
            t.turn(72); // 360 / 5 = 72 degrees
        }
    }

    public static void drawShape(Turtle t, int n, int pixels)
    {
        t.setColor(Color.black);
```

```
        // 4. Draw a polygon with n sides
        for(int i = 0; i < n; i++)
        {
            t.forward(pixels);
            t.turn(360 / n); // external angle
        }
    }

    public static void main(String[] args)
    {
        World world = new World(400, 400);
        Turtle yertle = new Turtle(world);
        yertle.penUp();
        yertle.moveTo(100, 200);
        yertle.penDown();
        yertle.setSpeed(25); // fast 0 - 100 slow

        // Draw a square
        drawSquare(yertle);
        // Move to another spot
        yertle.penUp();
        yertle.moveTo(250, 200);
        yertle.penDown();
```

```
// Draw a triangle
drawTriangle(yertle);
// Move to another spot
yertle.penUp();
yertle.moveTo(100, 50);
yertle.penDown();

// Draw a pentagon
drawPentagon(yertle);
// Move to another spot
yertle.penUp();
yertle.moveTo(250, 50);
yertle.penDown();

// Draw shapes of any number of sides and pixels
drawShape(yertle, 4, 50);

// 5. Draw a hexagon (6 sides) of length 20
yertle.penUp();
yertle.moveTo(200, 300);
yertle.penDown();
drawShape(yertle, 6, 20);
```

```
        world.show(true);
    }
}
```

2.8.4 - Summary

- (AP 2.8.A.1) A for loop is a type of iterative statement. There are three parts in a for loop header: the initialization (of the loop control variable or counter), the Boolean expression (testing the loop variable), and the update (to change the loop variable).
- (AP 2.8.A.2) In a for loop, the initialization statement is only executed once before the first Boolean expression evaluation. The variable being initialized is referred to as a loop control variable.
- (AP 2.8.A.2) The for loop Boolean expression is evaluated immediately after the **loop control variable** is initialized and then followed by each execution of the increment (or update) statement until it is false.
- (AP 2.8.A.2) In each iteration of the for loop, the update is executed after the entire loop body is executed and before the Boolean expression is evaluated again.
- (AP 2.8.A.3) A for loop can be rewritten into an equivalent while loop (and vice versa).

2.8.5 - AP Practice

- No notes to show

2.9 - Implementing Selection and Iteration Algorithms

- Implementation of algorithms and loops to solve problems.
- Standard algorithms that use loops and selection.

2.9.1 - Accumulator Pattern for Sum/Average

- The **accumulator pattern** is an algorithm that iterates through a set of values using a loop and updates an accumulator variable with said values.
- Used to compute a sum or average of a set of values.

2.9.2 - Input-Controlled Loop

- **Input-controlled loops** calculate the average of positive numbers using the accumulator pattern with a while loop. The number -1 is a **sentinel value**.
    - Sentinel values are indicators for a program to stop.

2.9.3 - Loop with if and Minimum/Maximum

- Common variation of the accumulator pattern is to have an if statement inside the loop that tests each value being considered in the loop.
    - Common in FRQ #1 for AP Exam.
- Use a variable to store the current minimum/maximum value to determine said minimum/maximum in a sequence of numbers.

2.9.4 - Divisibility

- To determine if an integer is evenly divisible by another → use remainder operator.

2.9.5 - Finding Digits with / and %

- Use division and remainder to find the digits of a number by dividing by 10.
    - Remember that the result truncates everything to the right of the decimal point.
- Remainder operator frequently used in the AP CSA exam.

2.9.6 - Math.random() in if Statements

- Math.random() returns a random number between 0.0 and 1.0 → simulates coin flip or an event occurring a certain percentage of time.

2.9.7 - Frequency

- Use a counter variable inside a loop → determine frequency with which specific criteria is met.
- AP Exam often gives a boolean method for students to use to determine some criteria in a set of values.

2.9.8 - Coding Challenge : Prime Number Finder

Java

```java
public class FindPrimeNumbers {

    public static boolean isPrime(int n) {

        if (n < 2) return false;

        for (int i = 2; i <= Math.sqrt(n); i++) {

            if (n % i == 0) return false;

        }

        return true;

    }


    public static int findPrimes(int num1, int num2) {

        int count = 0;

        for (int i = num1; i <= num2; i++) {

            if (isPrime(i)) {

                System.out.println(i);

                count++;

            }

        }
```

```
        return count;
    }

    public static void main(String[] args) {
        System.out.println("There are " + findPrimes(2, 100) + " prime numbers
between 2 and 100.");
    }
}
```

2.9.9 - Summary

- (AP 2.9.1) There are standard algorithms that use loops and selection to:
    - compute a sum or average of a set of values
    - determine a minimum or maximum value
    - identify if an integer is or is not evenly divisible by another integer
    - identify the individual digits in an integer
    - determine the frequency with which a specific criterion is met.
- The accumulator pattern is an algorithm that involves storing and updating an accumulator value within a loop, for example to compute a sum or average of a set of values.
- A common algorithm pattern is an if statement within a loop to test each value against a criterion, such as finding the minimum or maximum value in a sequence of numbers.

2.10 - Implementing String Algorithms

- Loops are often used for **String Traversals** or **String Processing** algorithms → code steps through a string character by character.
- Standard string algorithms can do:
    - Find one or more properties inside a substring.

    - Determine the number of substrings that meet a criteria.
    - Create new strings with the characters reversed.
- Remember that strings start at **index** 0.
    - The last character is **length()** - 1.
- AP CSA most frequently used string methods:
    - **int length() :** returns the number of characters in a String object.
    - **int indexOf(String str) :** returns the index of the first occurrence of str or -1 if str is not found.
    - **String substring(int from, int to) :** returns the substring beginning at index from and ending at index (to – 1). Note that s.substring(i,i+1) returns the character at index i.
    - **String substring(int from) :** returns substring(from, length()).

2.10.1 - While Find and Replace Loop

- While loop can be used with the String indexOf method to find certain characters in a string and process them.

2.10.2 - For Loops: Reverse String

- For loops can also be used to process strings, especially in situations where you know you will visit every character.
    - Usually start at 0 and use length() as ending condition.

2.10.3 - Coding Challenge : String Replacement Cats and Dogs

```java
public class ChallengeReplace {

    public static void main(String[] args) {

        String message = "I love cats! I have a cat named Coco. My cat's very smart! Dogs are great too, but I prefer cats and catnip.";

        int index = 0;
```

```
int count = 0;


while (message.indexOf("cat") >= 0) {
    index = message.indexOf("cat");
    String firstPart = message.substring(0, index);
    String lastPart = message.substring(index + 3);
    message = firstPart + "dog" + lastPart;
    count++;
}


System.out.println("After replacing 'cat' with 'dog':");
System.out.println(message);
System.out.println("Total replacements: " + count);


int fromIndex = 0;
while (message.indexOf("dogs", fromIndex) >= 0) {
    int dogsIndex = message.indexOf("dogs", fromIndex);
    String firstPart = message.substring(0, dogsIndex + 4);
    String lastPart = message.substring(dogsIndex + 4);
    message = firstPart + " and cats" + lastPart;
    fromIndex = dogsIndex + 8;
}
```

```
        System.out.println("After adding 'and cats' after 'dogs':");
        System.out.println(message);
    }
}
```

2.11 - Nested Iteration

- **Nested iteration statements** (**nested loops**) are statements that appear within bodies of other statements.

2.11.1 - Nested Loops with Turtles

- No Notes to show.

2.11.2 - Coding Challenge : Turtle Snowflakes

```
Java
import java.awt.*;
import java.util.*;

public class TurtleSnowflakes
{
    public static void main(String[] args)
    {
        World world = new World(400, 400);
        Turtle yertle = new Turtle(world);
        yertle.setSpeed(50);
```

```
        int turnAmount = 30;   // angle between each polygon
        int n = 3;             // number of sides for polygon (try 4 for square, 5
for pentagon, etc.)

        // outer loop runs enough times to go 360 degrees total
        for (int i = 0; i < 360 / turnAmount; i++)
        {
            // change color depending on even/odd iteration
            if (i % 2 == 0)
                yertle.setColor(Color.BLUE);
            else
                yertle.setColor(Color.CYAN);

            // draw polygon (inner loop)
            for (int side = 0; side < n; side++)
            {
                yertle.forward(50);
                yertle.turn(360 / n);  // use formula for polygon angle
            }

            // turn to position for next polygon
            yertle.turn(turnAmount);
```

```
        }


        world.show(true);
    }
}
```

2.11.3 - Summary

- Nested iteration statements are iteration statements that appear in the body of another iteration statement.
- When a loop is nested inside another loop, the inner loop must complete all its iterations before the outer loop can continue.

2.12 - Informal Runtime Analysis of Loops

- Practice tracing through code to determine the amount of iteration for each loop.

2.12.1 - Tracing Loops

- No notes to show.

2.12.2 - Counting Loop Iterations

- Loops can be analyzed by how many times they run → **run-time analysis** or a **statement execution count**.
    - The **statement execution count** indicates the number of times a statement is executed by a program.
        - Often calculated informally.
- For nested loops, you will multiply the amount of times the nested loop will run by the **times** the outside loop will run.

2.12.3 - Coding Challenge : POGIL Analyzing Loops

- No notes to show.

2.14 - Summary

- (AP 2.12.A.1) A statement execution count indicates the number of times a statement is executed by the program. Statement execution counts are often calculated informally through tracing and analysis of the iterative statements.
- A trace table can be used to keep track of the variables and their values throughout each iteration of the loop.
- The number of times a loop executes can be calculated by largestValue - smallestValue + 1 where these are the largest and smallest values of the loop counter variable possible in the body of the loop.
- The number of times a nested for-loop runs is the number of **times** the outer loop runs times the number of times the inner loop runs.
- In non-rectangular loops, the number of times the inner loop runs can be calculated with the sum of natural numbers formula n(n+1)/2 where n is the number of times the outer loop runs or the maximum number of times the inner loop runs.

Unit 3 - Class Creation

3.1 - Abstraction and Program Design

- **Classes** and **objects are an essential** part of Java → Turtle class.
    - Implementation and designing classes are heavily tested.

3.1.1 - Object-Oriented Design

- **Object-oriented design** (OOD) → deciding what classes are needed to solve problems.
    - Look for **nouns** in specification → designed into methods.
    - **Student** and **Course** are good nouns for classes.

3.1.2 - UML Class Diagrams

- Formal way of depicting classes and relative diagrams → **Unified Modeling Language** (UML).
    - Representation of procedural abstraction alongside behaviors and instance variables of the class.
    - **Class diagrams** represent this → not on AP CSA exam however.

3.1.3 - Abstraction

- **Abstraction** is the process of reducing complexity by focusing on the main idea.
    - Hides data irrelevant + focuses on good ideas → allows for documentation.

3.1.4 - Data Abstraction

- **Data abstraction** provides separation of abstract properties of data with concrete details of its representation.
    - Examples include the Turtle class.
    - Shows the main idea of the class.
- An **attribute** is a type of data abstraction.
- An **instance variable** is an attribute whose value is unique to each instance of the class → based on pre-defined attributes.
- A **class variable** is an attribute shared by all instances of the class.

3.1.5 - Procedural Abstraction

- **Procedural abstraction** is a name for a process that allows a method to be used only knowing what it does, not how it does it.
    - Through **method decomposition** → programmer breaks down larger behaviors of class into smaller behaviors.

        - Organization + reducing complexity + reusing code + maintainability and debugging.
    - Programmers can change internals of a method without notifying method users of the change as long as method signature and what it does is preserved/
    - Repetition of code is something to be avoided → procedural abstraction shares features.
    - Parameters add even more abstraction and flexibility → generalizations.

3.1.6 - Group Challenge : Game Design

- No notes to show.

3.1.7 - Summary

- (AP 3.1.A.1) **Abstraction** is the process of reducing complexity by focusing on the main idea. By hiding details irrelevant to the question at hand and bringing together related and useful details, abstraction reduces complexity and allows one to focus on the idea.
- (AP 3.1.A.2) **Data abstraction** provides a separation between the abstract properties of a data type and the concrete details of its representation. Data abstraction manages complexity by giving data a name without referencing the specific details of the representation. Data can take the form of a single variable or a collection of data, such as in a class or a set of data.
- (AP 3.1.A.3) An **attribute** is a type of data abstraction that is defined in a class outside any method or constructor. An **instance variable** is an attribute whose value is unique to each instance of the class. A **class variable** is an attribute shared by all instances of the class.
- (AP 3.1.A.4) **Procedural abstraction** provides a name for a process and allows a method to be used only knowing what it does, not how it does it. Through **method decomposition**, a programmer breaks down larger behaviors of the class into smaller behaviors by creating methods to represent each individual smaller behavior. A procedural abstraction may extract shared features to generalize

functionality instead of duplicating code. This allows for code reuse, which helps manage complexity.

- (AP 3.1.A.5) Using parameters allows procedures to be generalized, enabling the procedures to be reused with a range of input values or arguments.
- (AP 3.1.A.6) Using procedural abstraction in a program allows programmers to change the internals of a method (to make it faster, more efficient, use less storage, etc.) without needing to notify method users of the change as long as the method signature and what the method does is preserved.
- (AP 3.1.A.7) Prior to implementing a class, it is helpful to take time to design each class including its attributes and behaviors. This design can be represented using natural language or diagrams.

3.2 - Impact of Program Design

- Computer science follows the ACM professional code of ethics.
- Sometimes, programs have unintended consequences → difficult to ensure **system reliability** which refers to a program being able to continuously perform expected conditions without failure.
    - Programmers should make an effort to maximize system reliability by testing the program with a variety of conditions.
- Creation of programs has impacts on society, economy, and culture.
- Programmers often reuse code and publish it as **open source** → free to use.
    - If not open source, programmers need to obtain permission/purchase code before integrating it into their program.
- Fields of **AI (Artificial Intelligence)** and **Machine Learning** pose ethical questions.

3.2.1 - Groupwork: Impacts of CS

- No notes to show.

3.2.2 - Summary

- (AP 3.2.A.1) **System reliability** refers to the program being able to perform its tasks as expected under stated conditions without failure. Programmers should make an

effort to maximize system reliability by testing the program with a variety of conditions.

- (AP 3.2.A.2) The creation of programs has impacts on society, the economy, and culture. These impacts can be both beneficial and harmful. Programs meant to fill a need or solve a problem can have unintended harmful effects beyond their intended use.
- (AP 3.1.A.3) Legal issues and intellectual property concerns arise when creating programs. Programmers often reuse code written by others and published as open source and free to use. Incorporation of code that is not published as **open source** requires the programmer to obtain permission and often purchase the code before integrating it into their program.

3.3 - Anatomy of a Java Class

- We learned built-in **classes** and **objects**, but now we'll make our own.
- A **class** defines a blueprint for creating objects.
    - When creating **objects**, you create **instances** of that class and what you can do with those classes is determined by what methods are available.

3.3.1 - Parts of a Class

- Most classes start with the keyword public though not required.
    - Officially start with the class keyword, then the name of the class, then the body.
    - Names are almost always capitalized.
- Objects have both attributes and behaviors → **instance variables** and **methods**.
    - First thing in classes are usually instance variables since it is what differentiates objects between each other.
    - Next thing we define are **constructors** → initialize instance variables when an object is created → always designated public.
    - Next is **methods** → behaviors of the object of that class
        - Do things or return values.

- Three main anatomical features of a class are the **instance variables** which hold values for each object, **constructors** which initialize said values, and the **methods** that contain behaviors associated with those values.

3.3.2 - Designing a Class

- Important question →what data does it represent?
- How can we represent it? What behaviors should it have?

3.3.3 - Data Encapsulation

- **Data encapsulation** → implementation details of a class are kept hidden from external classes.
    - Private and public are **access modifiers** → affect how you can access classes, data, constructors, and methods.
    - Instance variables should be declared private to ensure data encapsulation.

3.3.4 - Instance Variables

- **Instance variables** (**attributes**, **fields**, **properties**) hold data for an object → each object has a copy of these variables.
    - In general and definitely on the AP CSA exam instance variables should be declared **private**.
    - Ensures data encapsulation so that implementation details are hidden and therefore makes it secure.
    - Keeps it easier to track how the program is working instead of changing the values of variables.

3.3.4 - Instance Methods

- **Instance methods** define what we actually do with an object → behaviors and functions.
    - Have direct access to instance variables + use them to perform tasks.
    - To call a method → use an object and then the dot operator alongside the name of the method.

3.3.6 - Coding Challenge : Virtual Pet Class

Java

```java
public class VirtualPet
{
    // instance variables
    private String name;
    private int health;
    private int happiness;  // extra variable from design

    // constructor written for you - do not change
    public VirtualPet(String initName, int initHealth)
    {
        // set instance variables
        name = initName;
        health = initHealth;
        happiness = 5; // default starting happiness
    }

    // Print the VirtualPet's data
    public void print()
    {
        System.out.println(name + " " + health + " " + happiness);
    }
```

```
    // Feed method to add to the health instance variable
    public void feed()
    {
        health += 1;
        System.out.println(name + " has been fed! Health is now " + health);
    }

    // Another method that changes an instance variable
    public void play()
    {
        happiness += 2;
        System.out.println(name + " played and is now happier! Happiness is " +
happiness);
    }

    // get method used for testing - do not change
    public int getHealth()
    {
        return health;
    }

    // main method for testing
```

```
    public static void main(String[] args)
    {
        VirtualPet p = new VirtualPet("Fluffy", 5);
        VirtualPet q = new VirtualPet("Spike", 3);

        p.feed();     // increase Fluffy's health
        p.play();     // increase Fluffy's happiness
        p.print();    // print Fluffy's current state

        q.feed();     // test other pet
        q.print();    // print Spike's current state
    }
}
```

3.3.7 - Design a Class for your Community

- No notes to show.

3.3.8 - Summary

- (AP 3.3.A.1) **Data encapsulation** is a technique in which the implementation details of a class are kept hidden from external classes.
- (AP 3.3.A.1) The keywords public and private affect the access of classes, data, constructors, and methods. The keyword private restricts access to the declaring class, while the keyword public allows access from classes outside the declaring class.
- (AP 3.3.A.2) In this course, classes are always designated public and are declared with the keyword class.

- (AP 3.3.A.3) In this course, constructors are always designated public.
- (AP 3.3.A.4) **Instance variables** belong to the object, and each object has its own copy of the variables.
- (AP 3.3.A.5) Access to attributes should be kept internal to the class in order to accomplish encapsulation. Therefore, it is good programming practice to designate the instance variables for these attributes as private unless the class specification states otherwise.
- **Instance Variables** define the attributes or data needed for objects, and **methods** define the behaviors or functions of the object.

3.3.9 - AP Practice

- No notes to show.

3.4 - Writing Constructors

- We learned how to create objects through **constructors**.

3.4.1 - Constructor Signature

- Constructors are usually written after the instance variables and before any methods.
- No return type, not even void, and it is always the same name as the class.
- Have a **parameter list** specified in the parentheses that declare the variables that will be used to hold arguments passed when the constructor is called.

3.4.2 - The Job of a Constructor

- Sets initial values for the object's instance variables to useful values.
    - "Useful"? We describe the values of all an object's instance variables at a given time as the object's **state**.
    - It is a **valid state** when all instance variables have values that let us use the object by invoking its public methods.
- An object's **state** refers to its attributes and their values at a given time and is defined by instance variables belonging to the object.
    - **Has-a** relationship between the object and its instance variables.

    - A person **has** a name, email, and phone number attributes.
- Easiest way to write a constructor is to not write one → **default constructor** with no parameters.
    - 0 for int, 0.0 for double, false for boolean, and null for all reference types.

3.4.3 - Coding Challenge : Student Class

```java
/**
 * class Student with 4 instance variables, a constructor, a print method, and a
 * main method to test them.
 */
public class Student
{
    // 4 instance variables
    private String name;
    private int age;
    private double gpa;
    private String major;

    // Constructor with 4 parameters
    public Student(String name, int age, double gpa, String major)
    {
        this.name = name;
        this.age = age;
        this.gpa = gpa;
```

```
        this.major = major;
    }

    // Print method that prints all instance variables
    public void print()
    {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("GPA: " + gpa);
        System.out.println("Major: " + major);
        System.out.println();
    }

    // main method
    public static void main(String[] args)
    {
        // Construct 2 Student objects using the constructor with different values
        Student s1 = new Student("Alice Johnson", 19, 3.8, "Computer Science");
        Student s2 = new Student("Brian Lee", 21, 3.5, "Mechanical Engineering");

        // call their print() methods
        s1.print();
        s2.print();
```

```
    }
}
```

3.4.3 - Design a Class for your Community

- No notes to show.

3.4.5 - Summary

- (AP 3.4.A.1) An object's **state** refers to its attributes and their values at a given time and is defined by instance variables belonging to the object. This defines a **has-a** relationship between the object and its instance variables.
- (AP 3.4.A.2) A constructor is used to set the initial state of an object, which should include initial values for all instance variables. When a constructor is called, memory is allocated for the object and the associated object reference is returned. Constructor parameters, if specified, provide data to initialize instance variables.
- A constructor must have the same name as the class! Constructors have no return type!
- (AP 3.4.A.4) When no constructor is written, Java provides a no-parameter constructor, and the instance variables are set to default values according to the data type of the attribute. This constructor is called the **default constructor**. (Note that AP 3.4.A.3 is covered in lesson 3.6).
- (3.4.A.5) Default values used by the default constructor:
    - The default value for an attribute of type int is 0.
    - The default value of an attribute of type double is 0.0.
    - The default value of an attribute of type boolean is false.
    - The default value of a reference type is null.

3.4.6 - AP Practice

- No notes to show.

3.5 - Methods: How to Write Them

- Three main parts of a class:
    - **Instance variables**, **constructors**, and **methods**.
    - We'll talk about methods in this topic.

3.5.1 - Defining and Calling Methods

- A **method** is a block of code that performs a specific task.
    - Defined inside a class, and can access instance variables of said class
- Three steps:
    - 1. **Object of the Class**: Declare the object in the main method or outside.
    - 2. **Method Call**: Whenever you want to use the method, objectName.methodName()
    - 3. **Method Definition**: Write the method's **header** and **body**.

3.5.2 - Void Methods

- A **void method** does not return a value → usually just print something.

3.5.3 - Non-void Methods

- A **non-void method** is a method that returns a single value → must have a **return** statement.
- Its header includes the **return type** in place of the keyword void.
- In non-void methods, we use **return by value** → return expression compatible with the return type is evaluated, and the value is returned.
    - Any code that is after the return statement is never executed.

3.5.4 - Accessors / Getters

- If you want code outside the class to access the value of an instance variable → write an **accessor method** (just called a **getter**).
    - Public method that takes no arguments and returns the value of a private instance variable.
    - Only return the value of the variable.

- Usually they are in separate classes. (in the example, it was in either the **Tester** or **Driver** class)

3.5.5 - toString

- Not strictly a getter, but it converts an object to string.
- This method will return a string that is printed out

3.5.6 - Mutators / Setters

- If you want to change values of instance variables → **mutator method** (just called a **setter**).
    - Void method with a name that starts with set and assigns a provided value to a given instance variable.
    - Not all instance variables are meant to be manipulated outside of a class.

3.5.7 - Parameters

- Setter methods usually contain parameters.
- A **parameter** is a variable in a method's header that is used to pass in data that the method needs to do its job.
    - New value that you want to assign to instance variables.
    - Receive values through those parameters and accomplish the method's task.
- An **argument** is a value that is passed into a method when the method is called.
    - Saved into a parameter variable.
    - Must be compatible in number and order with the types identified in the parameter list of the method signature.
    - **Call by value** - initializes parameters with copies of those arguments. Different from call by reference.

3.5.8 - Methods with Parameters that Return Calculated values

- Not all methods that return values are accessor/getter methods. Some methods have parameters and return values that are formed or calculated in a more complex algorithm.
    - May even use other functions or classes!

3.5.9 - Coding Challenge : Class Pet

Java

```java
/**
 * Pet class
 * Keeps track of animal patients for the Awesome Animal Clinic.
 * It stores and manages information such as name, age, weight, type, and breed.
 *
 * @author
 * @since 2025-10-27
 */
class Pet
{
    // ===== Instance Variables =====
    // Private variables to store pet attributes
    private String name;   // pet's name
    private int age;       // pet's age in years
    private double weight; // pet's weight in pounds
    private String type;   // type of animal (dog, cat, lizard, etc.)
    private String breed;  // breed of the animal


    // ===== Constructor =====
    /**
     * Constructs a Pet object and initializes all instance variables.
```

```
     * @param name   the name of the pet
     * @param age    the age of the pet
     * @param weight the weight of the pet
     * @param type   the type of animal
     * @param breed  the breed of the pet
     */
    public Pet(String name, int age, double weight, String type, String breed)
    {
        this.name = name;
        this.age = age;
        this.weight = weight;
        this.type = type;
        this.breed = breed;
    }

    // ===== Accessor (Getter) Methods =====
    public String getName() { return name; }
    public int getAge() { return age; }
    public double getWeight() { return weight; }
    public String getType() { return type; }
    public String getBreed() { return breed; }

    // ===== Mutator (Setter) Methods =====
```

```
    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setWeight(double weight) { this.weight = weight; }
    public void setType(String type) { this.type = type; }
    public void setBreed(String breed) { this.breed = breed; }

    // ===== toString Method =====
    /**
     * Returns a formatted string containing all the pet's information.
     */
    public String toString()
    {
        return "Name: " + name +
               "\nAge: " + age + " years" +
               "\nWeight: " + weight + " lbs" +
               "\nType: " + type +
               "\nBreed: " + breed + "\n";
    }
}

/**
 * TesterClass
 * Used to test the Pet class.
```

```
 */
public class TesterClass
{
    public static void main(String[] args)
    {
        // ===== Create 2 Pet objects =====
        Pet pet1 = new Pet("Buddy", 3, 25.4, "Dog", "Golden Retriever");
        Pet pet2 = new Pet("Whiskers", 2, 10.2, "Cat", "Siamese");

        // ===== Test accessor methods =====
        System.out.println("Testing Accessor Methods:");
        System.out.println(pet1.getName() + " is a " + pet1.getType());
        System.out.println(pet2.getName() + " weighs " + pet2.getWeight() + "
lbs\n");

        // ===== Test mutator methods =====
        System.out.println("Testing Mutator Methods:");
        pet1.setWeight(27.0);
        pet2.setAge(3);
        System.out.println(pet1.getName() + " now weighs " + pet1.getWeight() + "
lbs");
        System.out.println(pet2.getName() + " is now " + pet2.getAge() + " years
old\n");
```

```
        // ===== Test toString method =====
        System.out.println("Displaying Full Pet Information:");
        System.out.println(pet1.toString());
        System.out.println(pet2.toString());
    }
}
```

3.5.10 - Design a Class for your community

- No notes to show.

3.5.11 - Summary

- (AP 3.5.A.1) A void method does not return a value. Its header contains the keyword void before the method name.
- (AP 3.5.A.2) A **non-void method** returns a single value. Its header includes the return type in place of the keyword void.
- (AP 3.5.A.3) In non-void methods, a return expression compatible with the return type is evaluated, and the value is returned. This is referred to as **return by value**.
- (AP 3.5.A.4) The return keyword is used to return the flow of control to the point where the method or constructor was called. Any code that is sequentially after a return statement will never be executed. Executing a return statement inside a selection or iteration statement will halt the statement and exit the method or constructor.
- (AP 3.5.A.5) An **accessor method** (getter) allows objects of other classes to obtain a copy of the value of instance variables or class variables. An accessor method is a non-void method.
- (AP 3.5.A.6) A **mutator (modifier) method** (setter) is a method that changes the values of the instance variables or class variables. A mutator method is often a void method.
- Comparison of accessor/getters and mutator/setters syntax:

```
//Instance variable declaration
private typeOfVar varName;
```

```
/** setVar sets varName to a newValue
  * @param newValue
  */
public void setVarName(typeOfVar newValue)
{
      varName = newValue;
}
```

```
/** getVar() returns varName's value
  * @return varName
  */
public typeOfVar getVarName()
{
      return varName;
}
```

- (AP 3.5.A.7) Methods with parameters receive values through those parameters and use those values in accomplishing the method's task.
- (AP 3.5.A.8) When an argument is a primitive value, the parameter is initialized with a copy of that value. Changes to the parameter have no effect on the corresponding argument.
- The toString method is an overridden method that is included in classes to provide a description of a specific object. It generally includes what values are stored in the instance data of the object. If System.out.print or System.out.println is passed an object, that object's toString method is called, and the returned String is printed. An object's toString method is also used to get the String representation when concatenating the object to a String with the + operator.

3.5.12 - AP Practice

- No notes to show.

3.6 - Methods: Passing and Returning References of an Object

- Objects can be passed as arguments and returned from methods.
    - Powerful features of OOP → complex interactions.

3.6.1 - Objects as Instance Variables

- **Has-a** relationship; the object has an attribute associated with it.

- In java, each class is usually saved in its own file so that the main class is the file name.
- If you want to compile without changing the name, remove the public keyword.
    - Only the running class will have the main keyword and thus the file name.

3.6.2 - Objects as Arguments

- Java uses **call by value** → copy of argument will be saved onto the parameter.
    - If the value is changed, it does not change the original argument, only the copy.
    - Different from **call by reference** → DOES change the original argument.
- When a copy of a reference is passed and saved in the parameter variable → **aliases**.
    - Objects can have many names or aliases, but they ultimately are the same type of object.
    - Some classes are immutable → cannot be changed by any method.
    - Any new classes that you write have setters or methods that change the object → mutable.
        - If you pass in an object, you need to be careful that the method does not change it unless it is the intended behavior.

3.6.3 - Copying Parameter Objects

- When passing object references as parameters → references refer to the same object as references in the caller.
    - If it is immutable, it doesn't matter, but if it is **mutable**, then storing the passed-in reference in an instance variable leads to surprising results.
    - If some other code changes the object it will change for you too.
    - Make a copy just in case if that's not what you want.

3.6.4 - Parameter of the Same Class Type

- Methods cannot access private data and methods of a parameter that holds a reference to an object is the same type as the method's enclosing class.

3.6.5 - Returning Objects

- Methods can also return objects → only return the value of a variable.
    - If it is primitive, only a copy of the value is returned.
    - If it is an object reference → reference is returned, not a reference to a new copy of the object.
    - There can be multiple references of the same object.
    - If it is mutable → any changes to one reference can affect all other references.
- An **anonymous object** is when we create a new object without associating it with a variable.
    - Useful when you need it for a short time.

3.6.6 - Chaining Method Calls

- It may be useful to chain multiple method calls → important to know all return values of each object.

3.6.7 - Coding Challenge : Friends and Birthdays

```java
class Date
{
    // Instance variables
    private int month;
    private int day;
    private int year;

    // Constructor
    public Date(int month, int day, int year)
    {
```

```
        this.month = month;
        this.day = day;
        this.year = year;
    }

    // Copy constructor
    public Date(Date other)
    {
        this.month = other.month;
        this.day = other.day;
        this.year = other.year;
    }

    // Getters
    public int getMonth() { return month; }
    public int getDay() { return day; }
    public int getYear() { return year; }

    // Setter (example: allow updating the day)
    public void setDay(int newDay)
    {
        this.day = newDay;
    }
```

```
    // Optional: for easy display
    public String toString()
    {
        return month + "/" + day + "/" + year;
    }
}

public class Friend
{
    // Instance variables
    private String name;
    private Date birthdate;

    // Constructor (makes a copy of birthdate)
    public Friend(String name, Date birthdate)
    {
        this.name = name;
        this.birthdate = new Date(birthdate); // copy constructor
    }

    // Method to check if it's the friend's birthday
    public boolean isBirthday(Date today)
```

```
    {
        return (birthdate.getMonth() == today.getMonth() &&
                birthdate.getDay() == today.getDay());
    }

    // toString method
    public String toString()
    {
        return name + " (Birthday: " + birthdate + ")";
    }

    // Main method to test
    public static void main(String[] args)
    {
        // Create a Date object for your friend's birthday
        Date bday = new Date(10, 27, 2005);

        // Create a Friend object
        Friend myFriend = new Friend("Alex", bday);

        // Create a Date object for today's date
        Date today = new Date(10, 27, 2025);
```

```
        // Print friend info
        System.out.println(myFriend);

        // Check if today is the friend's birthday
        if (myFriend.isBirthday(today))
        {
            System.out.println("It's " + myFriend.name + "'s birthday today!");
        }
        else
        {
            System.out.println("It's not " + myFriend.name + "'s birthday
today.");
        }
    }
}
```

3.6.8 - Summary

- (AP 3.4.A.3) When a mutable object is a constructor parameter, the instance variable should be initialized with a copy of the referenced object. In this way, the instance variable does not hold a reference to the original object, and methods are prevented from modifying the state of the original object.
- (AP 3.6.A.1) When an argument is an object reference, the parameter is initialized with a copy of that reference; it does not create a new independent copy of the object. If the parameter refers to a mutable object, the method or constructor can use this reference to alter the state of the object. It is good programming practice to

not modify mutable objects that are passed as parameters unless required in the specification.

- (AP 3.6.A.2) When the return expression evaluates to an object reference, the reference is returned, not a reference to a new copy of the object.
- (AP 3.6.A.3) Methods cannot access the private data and methods of a parameter that holds a reference to an object unless the parameter is the same type as the method's enclosing class.

3.7 - Class (static) Variables and Methods

- Have already done **static** methods → wrote own main methods + others that were static.
    - Called **class methods** → belong to the class/not to any object.

3.7.1 - Class Methods

- Belong to the class and not to any object → Math methods w/one copy of the class.
- If you're calling from a different class + static → Class.method, if not static → object.method, if from the same class → method();
- Class methods can be either private or public → first word for any method definition.
- Class static methods can access/change values of class static variables and call class static methods.
    - Cannot access or change values of non-static instances/methods/etc.

3.7.2 - Class Variables

- **Class variables** belong to a class with all objects of a class sharing a single copy of the class variable.
    - Designated with the static keyword before variable type.
    - Accessed outside of the class by using the class name + dot operator.
    - Associated with a class, not objects of a class.

3.7.3 - final Keyword

- Keyword **final** can be used in front of a variable to make it a constant value that cannot be modified.
    - Traditionally capitalized.

3.7.4 - Coding Challenge : Static Song and counter

- No notes to show

3.7.5 - Summary

- (AP 3.7.A.1) Class methods cannot access or change the values of instance variables or call instance methods without being passed an instance of the class via a parameter.
- (AP 3.7.A.2) Class methods can access or change the values of class variables and can call other class methods.
- (AP 3.7.B.1) Class variables belong to the class, with all objects of a class sharing a single copy of the class variable. Class variables are designated with the **static** keyword before the variable type.
- (AP 3.7.B.2) Class variables that are designated public are accessed outside of the class by using the class name and the dot operator, since they are associated with a class, not objects of a class.
- (AP 3.7.B.3) When a variable is declared final, its value cannot be modified.

3.8 - Scope and Access

- **Scope** of a variable defined where a variable can be accessed/used.
    - Determined by the place of declaration. (look at closest enclosing curly braces)

3.8.1 - Class, Method, and Block Level Scope

- 3 levels of scope corresponding to different types of variables:
    - **Class level scope** → **instance variables** inside a class.
    - **Method level scope** → **local variables** (including **parameter variables**) inside a method.

- **Block level scope → loop variables**.

3.8.2 - Local Variables

- **Local variables** are defined in headers/bodies of blocks of code.
  - Only accessed in the block for which they are declared.
  - Inside a method + at top of the method.
  - Used within the method and don't exist outside of it.
- Parameter variables are also local variables that only exist for a method/constructor.
  - Cannot be declared public/private.
- Instance variables at class scope are shared by all methods → marked as public/private → have class scope no matter if public/private.

3.8.3 - Coding Challenge : Debugging

- No notes to show.

3.8.4 - Summary

- **Scope** is defined as where a variable is accessible or can be used.
- (AP 3.8.A.1) **Local variables** are variables declared in the headers or bodies of blocks of code. Local variables can only be accessed in the block in which they are declared.
- (AP 3.8.A.1) Parameters to constructors or methods are also considered local variables. These variables may only be used within the constructor or method and cannot be declared to be public or private.
- (AP 3.8.A.2) When there is a local variable or parameter with the same name as an instance variable, the variable name will refer to the local variable instead of the instance variable within the body of the constructor or method.

3.8.5 - AP Practice

- No notes to show.

3.9 - this Keyword

- The keyword “this” acts as a special variable holding a reference to the current object.

3.9.1 - this.instanceVariable

- “This” keyword is sometimes used to distinguish between variables.
    - Give instance variables the keyword; leave parameter variables as is.

3.9.2 - this as an Argument

- Can be used anywhere you would use an object variable + pass it to another method as an argument.

3.9.3 - Coding Challenge : Bank Account

- No notes to show.

3.9.4 - Summary

- (AP 3.9.A.1) Within an instance method or a constructor, the keyword acts as a special variable that holds a reference to the current object—the object whose method or constructor is being called.
- this.instanceVariable can be used to distinguish between this object’s instance variables and local parameter variables that may have the same variable names.
- (AP 3.9.A.2) The keyword this can be used to pass the current object as an argument in a method call.
- (AP 3.9.A.3) Class methods do not have a “this” reference.

3.9.5 - AP Practice

- No notes to show.

3.10 - Unit 3 Summary

3.10.1 - Concept Summary

- **Class** - A class defines a type and is used to define what all objects of that class know and can do.

- **Compiler** - Software that translates the Java source code (ends in .java) into the Java class file (ends in .class).
- **Compile time error** - An error that is found during the compilation. These are also called syntax errors.
- **Constructor** - Used to initialize fields in a newly created object.
- **Field** - A field holds data or a property - what an object knows or keeps track of.
- **Java** - A programming language that you can use to tell a computer what to do.
- **Main Method** - Where execution starts in a Java program.
- **Method** - Defines behavior - what an object can do.
- **Object** - Objects do the actual work in an object-oriented program.
- **Syntax Error** - A syntax error is an error in the specification of the program.

3.10.2 - Java Keyword Summary

- **class** - used to define a new class
- **public** - a visibility keyword which is used to control the classes that have access. The keyword public means the code in any class has direct access.
- **private** - a visibility keyword which is used to control the classes that have access. The keyword private means that only the code in the current class has direct access.

3.10.3 - Vocabulary Practice

- No notes to show

3.10.4 - Common Mistakes

- Forgetting to declare an object to call its methods.
- Forgetting to write get/set methods for private instance variables.
- Forgetting to write a constructor.
- Mismatch in name, number, type, order of arguments and return type between the method definition and the method call.
- Forgetting data types for every argument in the method definition.
- Forgetting to use what the method returns.

Unit 4 - Data Collections

4.1 - Ethical and Social Issues Around Data Collection

4.1.1 - Data Privacy

- Your phone has a lot of info + collects it to personalize decisions.
- Can sometimes be harmful/collect too much data.
- We must know the risks of sharing too much personal data.

4.1.2 - Data and Bias

- Fields of **AI (Artificial Intelligence)** and **Machine Learning (ML)** are growing and involve various ethical questions.
- **Algorithmic bias** describes systemic/repeated errors that creates unfair outcomes for a specific group of users → AI didn't even recognize black people in its early stages!
    - Programmers should be aware of their data set collection → make it as diverse and unbiased as possible.

4.1.3 - Summary

- (AP 4.1.A.1) When using a computer, personal privacy is at risk. When developing new programs, programmers should attempt to safeguard the personal privacy of the user.
- Computer use and the creation of programs have an impact on personal security and data privacy. These impacts can be beneficial and/or harmful.
- (AP 4.1.B.1) **Algorithmic bias** describes systemic and repeated errors in a program that create unfair outcomes for a specific group of users.
- (AP 4.1.B.2) Programmers should be aware of the data set collection method and the potential for bias when using this method before using the data to extrapolate new information or drawing conclusions.
- (AP 4.1.B.3) Some data sets are incomplete or contain inaccurate data. Using such data in the development or use of a program can cause the program to work incorrectly or inefficiently.
- (AP 4.1.C.1) Contents of a data set might be related to a specific question or topic and might not be appropriate to give correct answers or extrapolate information for a different question or topic.

4.2 - Data Sets

- A **data set** is a collection of specific pieces of information or data.
    - Can be manipulated + analyzed to solve a problem or answer a question.
    - While analyzing them → can be manipulated to answer a problem/question.
    - Values within the set are accessed/utilized one at a time and processed for a desired outcome.

4.2.1 - Summary

- (AP 4.2.A.1) A data set is a collection of specific pieces of information or data.
- (AP 4.2.A.2) Data sets can be manipulated and analyzed to solve a problem or answer a question. When analyzing data sets, values within the set are accessed and utilized one at a time and then processed according to the desired outcome.

- (AP 4.2.A.3) Data can be represented in a diagram by using a chart or table. This visual can be used to plan the algorithm that will be used to manipulate the data.

4.3 - Array Creation and Access

- Most programming languages have simple **data structures** for collection of data.
    - In Java and many programming languages → **array**.
- **Array** is a block of memory that stores a collection of data (**elements**) → useful for when you have many elements of data of the same type.
    - Use a number to access the array (called an **index**).
    - An array **index** is like a locker number → helps you find a specific datum within the data.
        - Starts at zero in java.

4.3.1 - Declaring and Creating an Array

- Arrays are **objects** → any variable that declares an array holds a reference to the object.
    - If it hasn't been created and you try to print it → will print **null**.
- Two ways to create an array → use keyword **new** to get memory or use an **initializer list** to set up values in the array.

4.3.2 - Using new to Create Arrays

- To create an empty array after declaring the variable → use the **new** keyword with the type and size of array. This creates the array in memory.

4.3.3 - Initializer Lists to Create Arrays

- Another way to create it is to use an **initializer list** → set the values in the array to a list of values in curly braces.
- When creating an array of a **primitive type** with initial values specified → space is allocated for the specified number of items of that type + values of array.
- When creating an array of an **object type** (String) with initial values → space is set aside for that number of object references. Objects are created + object references set so that the objects can be found.

4.3.4 - Array length

- Arrays know their **length** (amt. of elements).
    - The length/size of an array is established at time of creation and can't be changed.
    - Accessed through length attribute → public final instance variable that cannot be changed.
    - Use **dot-notation** to access the instance variable such as array.length

4.3.5 - Access and Modify Array Values

- To access items in an array → use an indexed **array variable** which is the array name and the index inside of a square bracket.
    - **Index** is a number that indicates the position of an item in a list.
    - A way to access is **arrayname[index]**.
- You can use **parallel arrays** to keep track of two things at once → corresponding indices.
- A powerful tool in the array **data abstraction** is that we can use variables for indexes.

4.3.6 - Coding Challenge : Countries Array

- No notes to show.

4.3.7 - Design an Array of Objects for your Community

- No notes to show.

4.3.8 - Summary

- (AP 4.3.A.1) An **array** stores multiple values of the same type. The values can be either primitive values or object references.
- (AP 4.3.A.2) The length (size) of an array is established at the time of creation and cannot be changed. The length of an array can be accessed through the length attribute.

- (AP 4.3.A.3) When an array is created using the keyword new, all of its elements are initialized to the default values for the element data type. The default value for int `` is ``0, for double is 0.0, for boolean is false, and for a reference type (like String or a class you have created) is null.
- (AP 4.3.A.4) Initializer lists can be used to create and initialize arrays.
- (AP 4.3.A.5) Square brackets [] are used to access and modify an element in a 1D (one dimensional) array using an index.
- (AP 4.3.A.6) The valid index values for an array are 0 through one less than the length of the array, inclusive. Using an index value outside of this range will result in an ArrayIndexOutOfBoundsException.

4.3.9 - AP Practice

- No notes to show.

4.3.10 - Arrays Game

- No notes to show.

4.4 - Array Traversals

- **Traversing** an array means visiting each element of the array → iteration.

4.4.1 - Index Variables

- No notes

4.4.2 - Loops to Traverse Arrays

- **Traversing an array** or **iteration** means the repetition of a statement for all elements of an array.
    - Start the index at **0** and end it at the **length** of the array.
    - Using a variable as the **index** → powerful **data abstraction**.

4.4.3 - Arrays as Objects and Parameters

- Arrays are objects in java. Array variables are referenced to an address in memory.
    - When passed → name of array refers to its address in memory.

- Any changes to the array in the method affects the original array.

4.4.4 - Looping through Part of an Array

- You can loop through a portion of the array by changing the index variable.
- Can also change the starting/ending conditions

4.4.5 - Common Errors When Looping Through An Array

- **ArrayIndexOutOfBoundsException** → index out of range; known as **off by one** error.

4.4.6 - Enhanced For Loop (For-Each) for arrays

- A special kind of loop called an **enhanced for loop** or **for each loop**.
  - Much easier to write as it does not involve an index variable → set to each value in the array successively.
  - To set it up → use **for (type variable: arrayname)**.

4.4.7 - Enhanced For Loop Limitations

- They cannot change the value stored in arrays, therefore incrementation/decrementation of all elements is impossible.

4.4.8 - Traversing Arrays of Objects

- Both index for loops and enhanced for loops can traverse an array of objects.\

4.4.9 Coding Challenge : SpellChecker

- No notes to show.

4.4.10 - Design an Array of Objects for your Community

- No notes to show.

4.4.11 - Summary

- (AP 4.4.A.1) **Traversing an array** is when repetition statements are used to access all or an ordered sequence of elements in an array.

- (AP 4.4.A.2) Traversing an array with an indexed for loop or while loop requires elements to be accessed using their indices.
- In for and while loops, make sure the index for an array starts at 0 and ends at the number of elements − 1. **Off by one** errors are easy to make when traversing an array, resulting in an **ArrayIndexOutOfBoundsException** being thrown.
- An **enhanced for loop**, also called a **for each loop**, can be used to loop through an array without using an index variable.
- To set up a for-each loop, use **for (type variable : arrayname)** where the type is the type for elements in the array, and read it as “for each variable value in arrayname”.
- (AP 4.4.A.3) An enhanced for loop header includes a variable, referred to as the enhanced for loop variable. For each iteration of the enhanced for loop, the enhanced for loop variable is assigned a copy of an element without using its index.
- (AP 4.4.A.4) Assigning a new value to the enhanced for loop variable does not change the value stored in the array. (So, you can’t change an array using the enhanced for loop.)
- (AP 4.4.A.5) When an array stores object references, the attributes can be modified by calling methods on the enhanced for loop variable. This does not change the object references stored in the array. (So, you can change the attributes of an object in an array using the enhanced for loop.)
- (AP 4.4.A.6) Code written using an enhanced for loop to traverse elements in an array can be rewritten using an indexed for loop or a while loop.

4.4.12 - Arrays Game

- No notes to show.

4.5 - Implementing Array Algorithms

- There are certain algorithms that must be understood for the AP Exam.

4.5.1 - Accumulator Pattern for Sum/Average

- The **accumulator pattern** is an algorithm that iterates through a set of values using a loop and updates an accumulator variable with those values.

- It has 4 steps:
    - Initialize the accumulator variable before the loop.
    - Loop through the values.
    - Update the accumulator variable inside the loop.
    - Print or use the accumulated value when the loop is done

4.5.2 - Min, Max, Search Algorithms

- The min/max search algorithms follow a similar principle than the previous activity's challenge.

4.5.3 - Looping From Back to Front

- You don't have to loop an array from beginning to end. You can also start at the end and move toward the front each time through the loop.
- The **return** statement is usually used to indicate when a certain condition has been met within traversals.

4.5.4 - Test Property

- We often loop through arrays to check if they have a property → ex. At least one is negative.
- Often given as a boolean method used to:
    - Determine if at least one element has a particular property
    - Determine if all elements have a particular property
    - Determine the number of elements having a particular property

4.5.5 - Pairs and Duplicates in Array

- Usually we use the indices [i] == [i + 1] to check for duplicates.

4.5.6 - Rotating Array Elements

- Rotating an array means shifting the array left or right by one position.

4.5.7 - Reversing an Array

- Reversing it is similar to rotating it → copy 1st element to last position, 2nd to second-to-last, etc.
    - Use a temporary variable to store the value of an element to overwrite.
    - Usually we need an indexed loop to access the elements at different indices unless we use a temporary array.

4.5.8 - Review and FRQ Practice for Arrays

- No notes to show.

4.6 - Using Text Files

- Files are used to store data in software used everyday.
- A **file** is storage for data that persists when the program is not running.
    - Can be retrieved during program execution.

4.6.1 - Java File, Scanner, and IOException Classes

- A file can be connected to the program using the File/Scanner classes.
    - Must be imported in from libraries → Scanner class is in the java.util package and file is in the java.io package (**input/output**)
- The Scanner class can be used to read in data from the file line by line after opening a file.
    - Takes a file object as an argument to create an input stream.
- If misspell → FileNotFoundException by Scanner constructor.
    - General error when input doesn't match expectations.
- Exceptions are ways for java to handle runtime errors during program execution.
    - When an exception is thrown → program stops executing and the exception is thrown back to the calling method.
- The **throws IOException** statement is added to the end of the method header.
- Java forces programmers to handle exceptions because runtime errors crash programs.
    - A better way of handling exceptions is the try/catch block statement.
    - Similar to if/else that is used for error handling.

4.6.2 - Reading in Data with Scanner

- Once a file is opened → data can be read using Scanner methods.
- List of methods used in the Scanner file:
    - Scanner(File f) the Scanner constructor that accepts a File for reading.
    - String nextLine() returns the next line of text up until the end of the line as a String read from the file or input source; returns null if there is no next line.
    - String next() returns the next String up until a white space is read from the file or input source.
    - int nextInt() returns the next int read from the file or input source. If the next int does not exist, it will result in an InputMismatchException. Note that this method does not read the end of the line, so the next call to nextLine() will return the rest of the line which will be empty.
    - double nextDouble() returns the next double read from the file or input source. If the next double does not exist, it will result in an InputMismatchException.
    - boolean nextBoolean() returns the next Boolean read from the file or input source. If the next boolean does not exist, it will result in an InputMismatchException.
    - boolean hasNext() returns true if there is a next item to read in the file or input source; false otherwise.
    - void close() closes the input stream.
- Scanner class can be used to read input from the keyboard or from a file.
- Whitespace can usually be dealt with using nextLine();

4.6.3 - Loop to Read in a File

- A while loop is usually used to read in a file with multiple lines.
    - Can use the method hasNext as the loop condition to detect if the file still contains elements to read.

4.6.4 - Save File Data into an Array

- We can save a file line by line into an array. However, we have to know the number of lines in the file to declare an array of the right size.
- Eventual data structures no longer need the size of the data in advance.

4.6.5 - Split Strings

- A **delimiter** is a character like a comma or a space that separates the units of data.
    - Returns a String array where each element in the array represents a field of data from the line.

4.6.6 - Object-Oriented Design with CSV files

- No notes to show.

4.6.7 - Coding Challenge : Array of pokemon from input File

- No notes to show.

4.6.8 - Optional Challenge with a Dataset

- No notes to show.

4.6.9 - Summary

- (AP 4.6.A.1) A **file** is storage for data that persists when the program is not running. The data in a file can be retrieved during program execution.
- (AP 4.6.A.2) A file can be connected to the program using the File and Scanner classes.
- (AP 4.6.A.3) A file can be opened by creating a File object, using the name of the file as the argument of the constructor. File(String str) is the File constructor that accepts a String file name to open for reading, where str is the pathname for the file.
- (AP 4.6.A.4) When using the File class, it is required to indicate what to do if the file with the provided name cannot be opened. One way to accomplish this is to add throws IOException to the header of the method that uses the file. If the file name is invalid, the program will terminate.

- (AP 4.6.A.5) The File and IOException classes are part of the java.io package. An import statement must be used to make these classes available for use in the program.
- (AP 4.6.A.6) The following Scanner methods and constructor—including what they do and when they are used—are part of the Java Quick Reference (p. 2) provided during the AP CSA exam:
    - Scanner(File f) the Scanner constructor that accepts a File for reading.
    - int nextInt() returns the next int read from the file or input source. If the next int does not exist, it will result in an InputMismatchException. Note that this method does not read the end of the line, so the next call to nextLine() will return the rest of the line which will be empty.
    - double nextDouble() returns the next double read from the file or input source. If the next double does not exist, it will result in an InputMismatchException.
    - boolean nextBoolean() returns the next Boolean read from the file or input source. If the next boolean does not exist, it will result in an InputMismatchException.
    - String nextLine() returns the next line of text up until the end of the line as a String read from the file or input source; returns the empty string if called immediately after another Scanner method like nextInt that is reading from the file or input source;returns null if there is no next line.
    - String next() returns the next String up until a white space is read from the file or input source.
    - boolean hasNext() returns true if there is a next item to read in the file or input source; false otherwise.
    - void close() closes the input stream.
- (AP 4.6.A.7) Using nextLine and the other Scanner methods together on the same input source sometimes requires code to adjust for the methods' different ways of handling whitespace.
- (AP 4.6.A.8) The following additional String method—including what it does and when it is used—is part of the Java Quick Reference provided during the AP CSA Exam:

- String[] split(String del) returns a String array where each element is a substring of this String, which has been split around matches of the given expression del.
- For example, String[] data = line.split(","); splits a line from a csv file along the commas and saves the substrings into the array data.
- (AP 4.6.A.9) A while loop can be used to detect if the file still contains elements to read by using the hasNext method as the condition of the loop.
- (AP 4.6.A.10) A file should be closed when the program is finished using it. The close method from Scanner is called to close the file.

4.7 - Wrapper Classes - Integer and Double

- A **wrapper class** enclosed around a primitive data type + gives it an appearance.
    - Part of the java.lang package → imported by default into Java programs.
- Integer and Double class are both **wrapper classes** → used to convert strings to primitive data types + useful Scanner class w/input.

4.7.1 - Creating Integer and Double Objects

- Examples of creating integer/double objects:
    - Java 7
        - Integer i = new Integer(2);
        - Double d = new Double(3.5)
    - Java 9
        - Integer i = 2;
        - Double d = 3.5;
- Java can return the maximum integer value if you subtract one from the minimum value.
    - Called **underflow** → conversely, you will get the minimum if you add one to the maximum → **overflow**.
- You can access it through Integer.MAX_VALUE or Integer.MIN_VALUE
    - Integer.MIN_VALUE → -2^{31}
    - Integer.MAX_VALUE → $2^{31} - 1$
    - Note that they loop back if you try to add/subtract one.

4.7.2 - Autoboxing and Unboxing

- **Autoboxing** is the automatic conversion that the Java compiler makes between primitive types and corresponding object wrapper classes.
- **Unboxing** is the automatic conversion that the Java compiler makes from the wrapper class to the primitive type.

4.7.3 - Passing Strings to Numbers

- Integer/Double classes have methods called Integer.parseInt + Integer.parseDouble → used to convert strings to numbers.
    - Often used w/Scanner class to convert input (read in as String) into int or double → create arithmetic/logical expressions.
        - Logical expressions use relational operators like < and >.
    - static int parseInt(String s) → returns String argument as signed int.
    - static double parseDouble(String s) → returns String argument as signed double.

4.7.4 - Coding Challenge: Pokemon Speed

- No notes to show.

4.7.5 - Summary

- The Integer class and Double class are **wrapper classes** that create objects from primitive types.
- (AP 4.7.A.1) The Integer class and Double class are part of the java.lang package.
- (AP 4.7.A.1) An Integer object is immutable, meaning once an Integer object is created, its attributes cannot be changed. A Double object is immutable, meaning once a Double object is created, its attributes cannot be changed.
- (AP 4.7.A.2) **Autoboxing** is the automatic conversion that the Java compiler makes between primitive types and their corresponding object wrapper classes. This includes converting an int to an Integer and a double to a Double. The Java compiler applies autoboxing when a primitive value is:
    - passed as a parameter to a method that expects an object of the corresponding wrapper class
    - assigned to a variable of the corresponding wrapper class

- (AP 4.7.A.3) **Unboxing** is the automatic conversion that the Java compiler makes from the wrapper class to the primitive type. This includes converting an Integer to an int and a Double to a double. The Java compiler applies unboxing when a wrapper class object is:
    - passed as a parameter to a method that expects a value of the corresponding primitive type
    - assigned to a variable of the corresponding primitive type
- (AP 4.7.A.4) The following class Integer method—including what it does and when it is used—is part of the Java Quick Reference: static int parseInt(String s) returns the String argument as a signed int.
- (AP 4.7.A.5) The following class Double method—including what it does and when it is used—is part of the Java Quick Reference: static double parseDouble(String s) returns the String argument as a signed double.

4.8 - ArrayList and its Methods

- Arrays are not very flexible → size thereof will remain the same forever.
- ArrayList is a re-sizable list.
    - Also takes care of keeping track of items that have been added to the array.
    - Will create a new bigger array under covers → when needed to hold more items.
- ArrayList is **mutable** in size + contains object references → can be changed during run time by adding/removing objects from it.

4.8.1 - import java.util.ArrayList

- A **package** is a set of related classes.
    - Other packages like java.util provide classes only to be used either by **importing** or (more rarely) referring to them by their full name.
    - You can either import specifically → java.util.ArrayList, or import all classes within the package with a “wildcard” import → java.util.*

4.8.2 - Declaring and Creating ArrayLists

- To declare an ArrayList use ArrayList<Type> name, where Type → called **type parameter** is the type of objects stored in an ArrayList.
    - Programmers use letter E and call it the **generic type** of an element.
    - ArrayList<E>
- As with other reference types → declaring an ArrayList variable doesn't create an ArrayList object.
    - Creates a variable that refers to ArrayList/null.
    - To create a ArrayList → invoke constructor such as new ArrayList<String>()
    - ArrayList constructor ArrayList() constructs an empty list.
- Get the number of items in an ArrayList through size().
- You can create ArrayLists of integer/double values → use wrapper classes Integer/Double as type parameter because ArrayLists can only hold objects, not primitive values.
    - All primitive types must be **wrapped** in objects before added.
    - Normally dealt with using autoboxing.

4.8.3 - ArrayList Methods

- The following are ArrayList methods that you need to know for the AP CSA exam.
    - Included on the reference sheet.
    - E in the method headers below stands for type of element in ArrayList.
        - E can be any type.
- **int size()** returns the number of elements in the list
- **boolean add(E obj)** appends obj to the end of the list and returns true
- **E remove(int index)** removes the item at the index and shifts remaining items to the left (to a lower index)
- **void add(int index, E obj)** moves any current objects at index or beyond to the right (to a higher index) and inserts obj at the index
- **E get(int index)** returns the item in the list at the index
- **E set(int index, E obj)** replaces the item at index with obj

4.8.4 - size()

- Get the number of items in an ArrayList using size() method. ArrayList starts out with a size of 0.
    - ArrayList<String> list = new ArrayList<String>();
    - System.out.println(list.size());

4.8.5 - add(obj)

- Add values to an ArrayList using add(obj) → add the object to the end of the list.
- Primitive types usually converted to corresponding wrapper classes.
    - When you pull int value out of a list of Integers → **unboxing**.

4.8.6 - add(index, obj)

- Two different add methods in the ArrayList class.
    - add(obj) method adds the passed object to the end of the list.
    - add(index, obj) method adds passed objects to the specified index, but first moves over any existing values to higher indices to make room for the new object.

4.8.7 - remove(index)

- You can remove values from an ArrayList using the remove(index) method.
    - Removes and returns items at a given index.

4.8.8 - get(index) and set(index, obj)

- You can get the object at an index using obj = listName.get(index) + set the object at an index using listName.set(index, obj)
    - Both methods require index arguments to refer to an existing element of the list.
        - Index must be greater than or equal to 0 and less than the size() of the list.
    - ArrayLists use get and set methods instead of the index operator that we use with arrays: array[index]. This is because ArrayList is a class with methods, not built-in type w/special support like arrays.

4.8.9 - Comparing arrays and ArrayLists

- When to use arrays or arrayLists?
    - Use an array when you know the size and specific type.
    - Use arrayList when you want to store several items of the same type but don't know how many you'll need.

4.8.10 - Coding Challenge : FRQ Digits

- No notes to show.

4.8.11 - Summary

- **ArrayLists** are re-sizable lists that allow adding and removing items to change their size during run time.
- (AP 4.8.A.4) The ArrayList class is part of the java.util package. An import statement can be used to make this class available for use in the program.(import java.util.ArrayList or java.util.*).
- (AP 4.8.A.1) An ArrayList object is **mutable** in size and contains object references. (Mutable means that it can change by adding and removing items from it.
- (AP 4.8.A.2) The ArrayList constructor ArrayList() constructs an empty list (of size 0).
- (AP 4.8.A.3) Java allows the generic type ArrayList<E>, where the generic type E specifies the type of the elements. (Without it, the type will be Object). When ArrayList<E> is specified, the types of the reference parameters and return type when using its methods are type E.
- (AP 4.8.A.3) ArrayList<E> is preferred over ArrayList (which creates a list of type Objects). For example, ArrayList<String> names = new ArrayList<String>(); allows the compiler to find errors that would otherwise be found at run time.
- ArrayLists cannot hold primitive types like int or double, so you must use the wrapper classes Integer or Double to put numerical values into an ArrayList. However autoboxing usually takes care of that for you.
- (AP 4.8.A.6) The indices for an ArrayList start at 0 and end at the number of elements - 1.

- (AP 4.8.A.5) The following ArrayList methods, including what they do and when they are used, are part of the Java Quick Reference:
    - **int size()** : Returns the number of elements in the list
    - **boolean add(E obj)** : Appends obj to end of list; returns true
    - **void add(int index, E obj)** : Inserts obj at position index (0 <= index <= size), moving elements at position index and higher to the right (adds 1 to their indices) and adds 1 to size
    - **remove(int index)** — Removes element from position index, moving elements at position index + 1 and higher to the left (subtracts 1 from their indices) and subtracts 1 from size; returns the element formerly at position index
    - **E get(int index)** : Returns the element at position index in the list
    - **E set(int index, E obj)** : Replaces the element at position index with obj; returns the element formerly at position index.

4.9 - ArrayList Traversals

- Traversing an ArrayList is when iteration is used to access all or an ordered sequence of said array.
    - We can use while loops, indexed for loops, or enhanced for loops.

4.9.1 - Enhanced For Loop

- Use enhanced for loop to traverse all of these items in an ArrayList.
    - Cannot be used to add/remove elements.
    - Use regular while/for loop to modify the size.

4.9.2 - For Loops and IndexOutOfBounds Exception

- Use regular indexes for loop to process list elements accessed using an index.
    - ArrayList indices start at 0 just like array indices → using the get(index) method, not index operator [].

4.9.3 - While Loop

- No notes to show.

4.9.4 - Reading in Files with java.nio.file

- Java.nio.file provides a better + easier way to read in files → Files classes in this package have an readAllLines method.
    - Returns them as a list of strings once all are read.
    - Use **interfaces**, but not covered in AP CSA.

4.9.5 - Traversing an ArrayList of Student Objects

- You can put any kind of object into an ArrayList.

4.9.4 - Coding Challenge : FRQ Word Pairs

- No notes to show.

4.9.7 - Summary

- (AP 4.9.A.1) Traversing an ArrayList is when iteration or recursive statements are used to access all or an ordered sequence of the elements in an ArrayList.
- ArrayLists can be traversed with an enhanced for loop, a while loop, or a regular for loop using an index.
- (AP 4.9.A.2) Deleting elements during a traversal of an ArrayList requires the use of special techniques to avoid skipping elements (since remove moves all the elements above the removed index down.)
- (AP 4.9.A.3) Attempting to access an index value outside of its range will result in an IndexOutOfBoundsException. (The indices for an ArrayList start at 0 and end at the number of elements − 1).
- (AP 4.9.A.4) Changing the size of an ArrayList while traversing it using an enhanced for loop can result in a ConcurrentModificationException. Therefore, when using an enhanced for loop to traverse an ArrayList, you should not add or remove elements.

4.10 - Implementing ArrayList Algorithms

- Standard ArrayList algorithms that utilize traversals that can:
    - insert elements
    - delete elements

- determine a minimum or maximum value
- compute a sum or average
- determine if at least one element has a particular property
- determine if all elements have a particular property
- determine the number of elements having a particular property
- access all consecutive pairs of elements
- determine the presence or absence of duplicate elements
- shift or rotate elements left or right
- reverse the order of the elements

4.10.1 - Add/Remove Elements

- You should be able to trace through code that uses ArrayList methods.

4.10.2 - Min, max, Sum, Average

- No notes to show.

4.10.3 - Finding a property

- No notes to show.

4.10.4 - Pairs and Duplicates

- No notes to show.

4.10.5 - Shift/Rotate an ArrayList

- No notes to show.

4.10.6 - Reversing an ArrayList

- No notes to show.

4.10.7 - Multiple or Parallel Data Structures

- No notes to show.

4.10.8 - Summary

- (AP 4.10.A.1) There are standard ArrayList algorithms that utilize traversals to:
    - determine a minimum or maximum value
    - compute a sum or average
    - determine if at least one element has a particular property
    - determine if all elements have a particular property
    - determine the number of elements having a particular property
    - access all consecutive pairs of elements
    - determine the presence or absence of duplicate elements
    - shift or rotate elements left or right
    - reverse the order of the elements
    - insert elements
    - delete elements
- (AP 4.10.A.2) Some algorithms require multiple String, array, or ArrayList objects to be traversed simultaneously.

4.10.9 - Review and FRQ Practice with ArrayLists

- No notes to show.

4.11 - 2D Array Creation and Access

- You can store items in **two-dimensional** (2D) arrays that have both **rows** and **columns**.
    - A **row** has horizontal elements, a **column** has vertical elements.
- A **2D array** is stored as an array of arrays in java.
    - Similarly created/indexed to 1D array objects.
    - Established at the time of creation and cannot be changed.
    - Store either primitive data or object reference data.

4.11.1 - Array Storage

- Many programming languages store 2D arrays in 1D arrays.
    - Typically you do it row-by-row (**row-major** order), though some can be column-by-column. (**column-major** order)
- On exam assume that any 2D array is in row-major order.

- In the AP Exam there will be rigid inner-arrays all having the same size.
    - Can have **non-rectangular arrays** or **ragged arrays** that don't.

4.11.2 - Declaring 2D Arrays

```java
// 2D array declaration

// datatype[][] variableName = new datatype[numberRows][numberCols];

int[][] ticketInfo = new int[2][3];

String[][] seatingChart = new String[3][2];
```

- Arrays are objects in java that hold references to objects.
    - If there isn't any **new** → printing it would result in **null**.
    - When a 2D array is created using **new** → elements initialized to default values for the element data type. (int is 0, double is 0.0, boolean is false, reference type is null)

4.11.3 - Initializer Lists for 2D Arrays

- You can initialize 2D arrays; no need to specify the size as it is determined by the list itself.
- Use curly braces {} to denote initialization.

4.11.4 - Set Values in a 2D Array

- When accessing arrays, the first number is used for the row, and the second number is used for the column.
    - Use assignment statements with the name of the array followed by the row/column index with a value after the = sign.

4.11.5 - Get a Value from a 2D Array

- No notes to show. (same thing as above)

4.11.6 - 2D Array Row and Column Length

- Arrays are immutable + know their length.
    - Public read-only attribute for array objects.
    - number of rows can be accessed through array name and .length.
    - number of columns accessed through array name + [index] + length.
- Valid ranges for arrays are from 0 to one less than the number of rows, and from 0 to one less than the number of columns.

4.11.7 - Coding Challenge : ASCII Art

- No notes to show.

4.11.8 - Summary

- (AP 4.11.A.1) A **2D array** is stored as an array of arrays. Therefore, the way 2D arrays are created and indexed is similar to 1D array objects. The size of a 2D array is established at the time of creation and cannot be changed. 2D arrays can store either primitive data or object reference data. Nonrectangular 2D array objects (with varying column length for each row) are outside the scope of the AP Computer Science A course and exam.
- 2D arrays are declared and created with the following syntax: datatype[][] variableName = new datatype[numberRows][numberCols];
- (AP 4.11.A.2) When a 2D array is created using the keyword new, all of its elements are initialized to the default values for the element data type. The default value for int is 0, for double is 0.0, for boolean is false, and for a reference type is null.
- (AP 4.11.A.3) The initializer list used to create and initialize a 2D array consists of initializer lists that represent 1D arrays; for example, int[][] arr2D = { {1, 2, 3}, {4, 5, 6} };.
- (AP 4.11.A.4) The square brackets [row][col] are used to access and modify an element in a 2D array. For the purposes of the AP exam, when accessing the element at arr[first][second], the first index is used for rows, the second index is used for columns.

- **Row-major order** refers to an ordering of 2D array elements where traversal occurs across each row, while **column-major order** traversal occurs down each column.
- (AP 4.11.A.5) A single array that is a row of a 2D array can be accessed using the 2D array name and a single set of square brackets containing the row index.
- (AP 4.11.A.6) The number of rows contained in a 2D array can be accessed through the length attribute. The valid row index values for a 2D array are 0 through one less than the number of rows or the length of the array, inclusive. The number of columns contained in a 2D array can be accessed through the length attribute of one of the rows. The valid column index values for a 2D array are 0 through one less than the number of columns or the length of any given row of the array, inclusive. For example, given a 2D array named values, the number of rows is values.length and the number of columns is values[0].length`. Using an index value outside of these ranges will result in an ArrayIndexOutOfBoundsException.

4.11.9 - 2D Arrays Game

- No notes to show.

4.12 - 2D Array Traversals: Nested Loops

4.12.1 - Nested Loops

- Nested iteration statements traverse and access all or an ordered sequence of elements in a 2D array.
    - 2D arrays are stored as arrays of arrays → traversed during for loops and enhanced for loops similar to 1D objects.
- Down below is an example of a **nested for loop** for all elements in a 2D array.

```
C/C++

int[][] array = { {1,2,3},{4,5,6}};

for (int row = 0; row < array.length; row++)

{
```

```
    for (int col = 0; col < array[0].length; col++)
    {
         System.out.println( array[row][col] );
    }
}
```

4.12.2 - Row-Major and Column-Major Traversals

- Nested iteration statements can traverse a 2D array in row-major order, column-major order, or a uniquely defined order.
    - **Row-major order** refers to an ordering of 2D array elements where traversal is through row (more common).
    - **Column-major order** traversals occur down each column.

4.12.3 - Enhanced For-Each Loop for 2D Arrays

- Enhanced loops are much simpler to use since they don't require indices and brackets.
    - Only use them if you are not changing the values in an array of primitive types → val cannot be changed.

4.12.4 - 2D Array of Objects

- Photographs/images are made up of 2D arrays of **pixels**.

4.12.5 - Coding Challenge : Picture Lab

- No notes to show.

4.12.6 - Summary

- (AP 4.12.A.1) Nested iteration statements (loops) are used to traverse and access all or an ordered sequence of elements in a 2D array. Since 2D arrays are stored as

arrays of arrays, the way 2D arrays are traversed using for loops and enhanced for loops is similar to 1D array objects.

- (AP 4.12.A.1) Nested iteration statements can be written to traverse the 2D array in row-major order, column-major order, or a uniquely defined order. **Row-major order** refers to an ordering of 2D array elements where traversal occurs across each row, whereas **column-major order** traversal occurs down each column.
- (AP 4.12.A.2) The outer loop of a nested enhanced for loop used to traverse a 2D array traverses the rows. Therefore, the enhanced for loop variable must be the type of each row, which is a 1D array. The inner loop traverses a single row. Therefore, the inner enhanced for loop variable must be the same type as the elements stored in the 1D array. Assigning a new value to the enhanced for loop variable does not change the value stored in the array.
- The 2D array's length gives the number of rows. A row's length array[0].length gives the number of columns.

4.12.7 - AP Practice

- No notes to show.

4.12.8 - 2D Arrays and Loops Game

- No notes to show.

4.13 - Implementing 2D Array Algorithms

- All of the array algorithms can be applied to 2D arrays too. There are standard algorithms that utilize 2D array traversals to:
    - determine a minimum or maximum value of all the elements or for a designated row, column, or other subsection
    - compute a sum or average of all the elements or for a designated row, column, or other subsection
    - determine if at least one element has a particular property in the entire 2D array or for a designated row, column, or other subsection
    - determine if all elements of the 2D array or a designated row, column, or other subsection have a particular property

    - determine the number of elements in the 2D array or in a designated row, column, or other subsection having a particular property
    - access all consecutive pairs of elements
    - determine the presence or absence of duplicate elements in the 2D array or in a designated row, column, or other subsection
    - shift or rotate elements in a row left or right or in a column up or down
    - reverse the order of the elements in a row or column
- Examples of either nested loop or enhanced nested loops:

```java
// 1 Dimensional Array Traversal
for (int value : array)
{
    if (value ....)
        ...
}

for(int i=0; i < array.length; i++)
{
   if (array[i] ....)
       ...
}

// 2 Dimensional Array Traversal
for (int row = 0; row < array.length; row++)
{
```

```
    for (int col = 0; col < array[0].length; col++)
    {
        if (array[row][col] ....)
            ...
    }
}

// enhanced for loops
for (int[] row : array)
{
    for (int value : row)
    {
        if (value ....)
            ...
    }
}
```

4.13.1 - Sum, Average, Min, Max 2D Array Algorithms

- No notes to show.

4.13.2 - Subsection of a 2D array for a Property

- You can loop through just part of a 2D array → change the starting/ending value to loop through a subset of a 2D array.

4.13.3 - Duplicates in 2D Arrays

- You can determine the presence/absence of duplicate elements in the 2D array or in a designated row, column, or other subsection.

4.13.4 - Rotate and Reverse

- You can rotate/reverse the order of the elements in a row or column.

4.13.5 - Review and FRQ Practice with 2D Arrays

- No notes to show.

4.14 - Searching Algorithms

- The ability to find specific data → **searching**.
- For AP CSA you need to know **linear (sequential) search** and **binary search**.
    - **Linear search** is a standard algorithm that checks each element in order until a value is found.
    - **Binary search** can be used on **sorted** data → divides a section in-half by narrowing down the space complexity.

4.14.1 - Linear Search

- Used to find a value in *unsorted* data → usually a simple for loop.
    - Time complexity of O(n)

4.14.2 - Linear Search with 2D Arrays

- We can also apply a linear search algorithm to data in a 2D array.

4.14.3 - Binary Search

- The idea of **binary search** is iteratively cutting the searching space in-half.
    - Calculate the middle index as left + right / 2 → left starts out at 0 and right starts out at the array length - 1.
- The following is an AP CSA binarySearch.

Java

```java
public class SearchTest
{
    public static int binarySearch(int[] elements, int target)
    {
        int left = 0;
        int right = elements.length - 1;
        while (left <= right)
        {
            int middle = (left + right) / 2;
            if (target < elements[middle])
            {
                right = middle - 1;
            }
            else if (target > elements[middle])
            {
                left = middle + 1;
            }
            else
            {
                return middle;
            }
        }
        return -1;
```

```
    }

    public static void main(String[] args)
    {
        int[] arr1 = {-20, 3, 15, 81, 432};

        // test when the target is in the middle
        int index = binarySearch(arr1, 15);
        System.out.println(index);

        // test when the target is the first item in the array
        index = binarySearch(arr1, -20);
        System.out.println(index);

        // test when the target is in the array - last
        index = binarySearch(arr1, 432);
        System.out.println(index);

        // test when the target is not in the array
        index = binarySearch(arr1, 53);
        System.out.println(index);
    }
}
```

- Also used with String → use compareTo method rather than < or >.
    - **int compareTo(String other)** returns a negative value if the current string is less than the other string, 0 if they have the same characters, and a positive value if the current string is greater than the other string.

4.14.4 - Runtimes

- How do we choose between two algorithms to solve the same problem?
    - Different characteristics and **runtimes** → depends on data.
- Binary search is much faster than linear search → think of the **worst case behavior**.
    - Use big-O runtime notation, in which linear is O(n) and binary search is O(log n).

4.14.5 - Coding Challenge : Search Runtimes

- No notes to show.

4.14.6 - Summary

- (AP 4.14.A.1) **Linear search algorithms** are standard algorithms that check each element in order until the desired value is found or all elements in the array or ArrayList have been checked. Linear search algorithms can begin the search process from either end of the array or ArrayList.
- (AP 4.14.A.2) When applying linear search algorithms to 2D arrays, each row must be accessed then linear search applied to each row of the 2D array.
- The **binary search** algorithm starts at the middle of a sorted array or ArrayList and eliminates half of the array or ArrayList in each iteration until the desired value is found or all elements have been eliminated.
- (AP 4.17.B.1 preview) Data must be in sorted order to use the binary search algorithm.
- (AP 4.17.B.2 preview) Binary search is typically more efficient than linear search.
- (AP 4.17.B.3 preview) The binary search algorithm can be written either iteratively or recursively. (The recursive solution will be presented in lesson 4.17).
- Informal run-time comparisons of program code segments can be made using statement execution counts.

4.15 - Sorting Algorithms

- There are many sorting algorithms to put an array of elements in alphabetic or numerical order. The three most important ones are the following:
    - **Selection sort** repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it into its correct (and final) position in the sorted portion of the list.
    - **Insertion sort** inserts an element from the unsorted portion of a list into its correct (but not necessarily final) position in the sorted portion of the list by shifting elements of the sorted portion to make room for the new element.
    - **Merge sort** breaks the elements into two parts and recursively sort each part. An array of one item is sorted (base case). Then merge the two sorted arrays into one. MergeSort will be covered in lesson 4.17.

4.15.1 - Selection Sort

- Starts at index 0 and looks through the entire array → keeps track of the index with smallest value.
    - Swaps value with smallest index with index 0.
    - Has time complexity $O(n^2)$
- The following code is base code for selection sort.

```
Java

import java.util.Arrays;


public class SortTest

{

    public static void selectionSort(int[] elements)

    {

        for (int j = 0; j < elements.length - 1; j++)
```

```
        {
            int minIndex = j;
            for (int k = j + 1; k < elements.length; k++)
            {
                if (elements[k] < elements[minIndex])
                {
                    minIndex = k;
                }
            }
            int temp = elements[j];
            elements[j] = elements[minIndex];
            elements[minIndex] = temp;
        }
    }

    public static void main(String[] args)
    {
        int[] arr1 = {3, 86, -20, 14, 40};
        System.out.println(Arrays.toString(arr1));
        selectionSort(arr1);
        System.out.println(Arrays.toString(arr1));
    }
}
```

4.15.2 - Insertion Sort

- Starts at index 1 and inserts value at index 1 into its correct place in the already sorted part.
- Go through the list, and if a value is smaller than the previous value, insert it back.
    - If a value is smaller than multiple seen values, sort it back that amount.
- Below is an example of insertionSort.

```java
import java.util.Arrays;


public class SortTest
{
    public static void insertionSort(int[] elements)
    {
        for (int j = 1; j < elements.length; j++)
        {
            int temp = elements[j];
            int possibleIndex = j;
            while (possibleIndex > 0 && temp < elements[possibleIndex - 1])
            {
                elements[possibleIndex] = elements[possibleIndex - 1];
                possibleIndex--;
            }
            elements[possibleIndex] = temp;
```

```
        }
    }

    public static void main(String[] args)
    {
        int[] arr1 = {3, 86, -20, 14, 40};
        System.out.println(Arrays.toString(arr1));
        insertionSort(arr1);
        System.out.println(Arrays.toString(arr1));
    }
}
```

4.15.3 - Coding Challenge : Sort Runtimes

Test Case	Selection Sort Runtime	Insertion Sort Runtime
Random order: {5,3,4,2,1}	~$O(n^2)$	~$O(n^2)$
Reverse order: {5, 4, 3, 2, 1}	$O(n^2)$	$O(n^2)$
Almost sorted: {1, 2, 3, 5, 4}	~O(n)	~O(n)

4.15.4 - Summary

- (AP 4.15.A.1) Selection sort and insertion sort are iterative sorting algorithms that can be used to sort elements in an array or ArrayList.

- (AP 4.15.A.2) **Selection sort** repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it into its correct (and final) position in the sorted portion of the list.
- (AP 4.15.A.3) **Insertion sort** inserts an element from the unsorted portion of a list into its correct (but not necessarily final) position in the sorted portion of the list by shifting elements of the sorted portion to make room for the new element.
- Informal run-time comparisons of program code segments can be made using statement execution counts.

4.15.5 - Search/Sort MC Exercises

- No notes to show.

4.16 - Recursion

- **Recursion** is when a method calls itself → a form of repetition.

```java
public static void neverEnd()

{

  System.out.println("This is the method that never ends!");

  neverEnd();

}
```

- Recursion that doesn't stop (keeps calling itself) is denoted by **infinite recursion**.
    - Not useful → StackOverFlowError.

4.16.1 - Recursive Call

- Well-constructed recursive methods contain a **base case**, which halts the recursion, and one **recursive call**.

4.16.2 - Why use Recursion?

- Recursion is most useful when a problem can be broken up into smaller and similar problems.
- Also used to create fractals and infinite shapes.
- Any iterative code can be written recursively and vice versa.
    - Limits to how much recursion a program can do in languages.
- More powerful than simple loops → branching structures, or "trees" which are common in computer programs.

4.16.3 - Factorial Method

- The method below calculates the **factorial** of a number → defined as n * factorial(n-1).

```java
public static int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n-1);
}
```

4.16.4 - Base Case

- Every non-infinitive recursive method must have one **base case**.
    - Smallest possible problem that the method knows how to solve → directly without any recursion.
    - The goal for many recursive methods is to shrink the problem to base cases.
    - Usually the stopping point for a recursive method.

4.16.5 - Tracing Recursive Methods

- A **call stack** keeps track of methods that have been called since the main method executes.
    - A **stack** is a way of organizing data that adds/removes items only from the top of the stack.

4.16.6 - Tracing Challenge : Recursion

- No notes to show.

4.16.7 - Summary

- (AP 4.16.A.1) A **recursive method** is a method that calls itself.
- (AP 4.16.A.1) Recursive methods contain at least one **base case**, which halts the recursion, and at least one **recursive call**. (unless it is a case of **infinite recursion**).
- (AP 4.16.A.1) Recursion is another form of repetition.
- (AP 4.16.A.2) Each recursive call has its own set of local variables, including the parameters. Parameter values capture the progress of a recursive process, much like loop control variable values capture the progress of a loop.
- (AP 4.16.A.3) Any recursive solution can be replicated through the use of an iterative approach and vice versa. (although the recursive solution may have memory limitations for the recursive call stack, and the iterative approach may require additional data structures).
- Note that writing recursive code is outside the scope of the AP Computer Science A course and exam.
- Recursion can be used to traverse Strings, arrays, and ArrayLists.

4.17 Recursive Searching and Sorting

- We'll learn modifications of recursion → **recursive binary search** and **recursive merge-sort**. Both are sorting algorithms.

4.17.1 - Recursive Binary Search

- More efficient as it starts in the middle of the array + eliminates half of searching space per iteration.

- Only works on sorted data.
- Written as follows.

Java

```java
public class IterativeBinarySearch

{

    public static int binarySearch(int[] elements, int target)

    {

        int left = 0;

        int right = elements.length - 1;

        while (left <= right)

        {

            int middle = (left + right) / 2;

            if (target < elements[middle])

            {

                right = middle - 1;

            }

            else if (target > elements[middle])

            {

                left = middle + 1;

            }

            else

            {

                return middle;
```

```
            }
        }
        return -1;
    }

    public static void main(String[] args)
    {
        int[] arr1 = {-20, 3, 15, 81, 432};

        int index = binarySearch(arr1, 81);
        System.out.println(index);
    }
}
```

- **Recursive binary search** starts at the middle as well + eliminates half of array per recursive call.
- Written as follows

Java

```
public class RecursiveBinarySearch
{
    public static int recursiveBinarySearch(
        int[] array, int start, int end, int target)
    {
```

```
        int middle = (start + end) / 2;
        // base case: check middle element
        if (target == array[middle])
        {
            return middle;
        }
        // base case: check if we've run out of elements
        if (end < start)
        {
            return -1; // not found
        }
        // recursive call: search start to middle
        if (target < array[middle])
        {
            return recursiveBinarySearch(array, start, middle - 1, target);
        }
        // recursive call: search middle to end
        if (target > array[middle])
        {
            return recursiveBinarySearch(array, middle + 1, end, target);
        }
        return -1;
    }
```

```
    public static void main(String[] args)
    {
        int[] array = {3, 7, 12, 19, 22, 25, 29, 30};
        int target = 25;
        int foundIndex = recursiveBinarySearch(array, 0, array.length - 1,
target);
        System.out.println(target + " was found at index " + foundIndex);
    }
}
```

4.17.2 - Merge Sort

- **Merge sort** is purely recursive → sort elements in an array.
- More efficient than Selection Sort + Insertion Sort → divides the problem in half, a technique called **divide and conquer** algorithm.
    - Repeatedly divides arrays into smaller subarrays until they are one element.
    - Brings them back together to form the final sorted array.
- Example code:

```
Java
import java.util.Arrays;


public class SortTest
{
    public static void mergeSort(int[] elements)
```

```
{
    int n = elements.length;
    int[] temp = new int[n];
    mergeSortHelper(elements, 0, n - 1, temp);
}

private static void mergeSortHelper(
    int[] elements, int from, int to, int[] temp)
{
    if (from < to)
    {
        int middle = (from + to) / 2;
        mergeSortHelper(elements, from, middle, temp);
        mergeSortHelper(elements, middle + 1, to, temp);
        merge(elements, from, middle, to, temp);
    }
}

private static void merge(
    int[] elements, int from, int mid, int to, int[] temp)
{
    int i = from;
    int j = mid + 1;
```

```
        int k = from;


        while (i <= mid && j <= to)
        {
            if (elements[i] < elements[j])
            {
                temp[k] = elements[i];
                i++;
            }
            else
            {
                temp[k] = elements[j];
                j++;
            }
            k++;
        }


        while (i <= mid)
        {
            temp[k] = elements[i];
            i++;
            k++;
        }
```

```
        while (j <= to)
        {
            temp[k] = elements[j];
            j++;
            k++;
        }

        for (k = from; k <= to; k++)
        {
            elements[k] = temp[k];
        }
    }

    public static void main(String[] args)
    {
        int[] arr1 = {86, 3, 43, 5};
        System.out.println(Arrays.toString(arr1));
        mergeSort(arr1);
        System.out.println(Arrays.toString(arr1));
    }
}
```

4.17.3 - Tracing Challenge : Recursive Search and Sort

- No noets to show.

4.17.4 - Summary

- (AP 4.17.A.1) Recursion can be used to traverse String objects, arrays, and ArrayList objects.
- (AP 4.17.B.1) Data must be in sorted order to use the binary search algorithm. **Binary search** starts at the middle of a sorted array or ArrayList and eliminates half of the array or ArrayList in each recursive call until the desired value is found or all elements have been eliminated.
- (AP 4.17.B.2) Binary search is typically more efficient than linear search.
- (AP 4.17.B.3) The binary search algorithm can be written either iteratively or recursively.
- (AP 4.17.C.1) **Merge sort** is a recursive sorting algorithm that can be used to sort elements in an array or ArrayList.
- (AP 4.17.C.2) Merge sort repeatedly divides an array into smaller subarrays until each subarray is one element and then recursively merges the sorted subarrays back together in sorted order to form the final sorted array.

4.17.5 - Summary and Review Exercises

- No notes to show. Done with AP CSA content!